

Received 11 March 2024, accepted 20 May 2024, date of publication 15 July 2024, date of current version 23 July 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3428368

RESEARCH ARTICLE

A Reference Architecture Based on Reflection for Self-Adaptive Software: A Second Release

FRANK JOSÉ AFFONSO¹, GABRIEL NAGASSAKI CAMPOS¹,
AND GUILHERME GUGUER MENALDO²

¹Department of Statistics, Applied Mathematics and Computation, São Paulo State University (UNESP), Rio Claro, São Paulo 13506-900, Brazil

²IBM Corporation, São Paulo 13186-900, Brazil

Corresponding author: Frank José Affonso (f.affonso@unesp.br)

This work was supported in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES), Brazil, under Grant 88887.670715/2022-00; and in part by São Paulo Research Foundation (FAPESP), Brazil, under Grant 2017/01703-6 and Grant 2019/21510-3.

ABSTRACT The development of Self-adaptive Software (SaS) is not a trivial task because this type of software has specific features compared to traditional ones. In short, SaS can reflect on its internal and external states and propose structural, behavioral, and contextual changes that can be incorporated at runtime. Manual adaptation tasks, even if very well executed, normally become onerous in time and effort, besides being error prone because of the involuntary injection of errors by the developers. Automated processes have been used as a feasible solution to conduct software adaptation at runtime by minimizing human involvement (e.g., software engineers and developers) and quickening up the execution of tasks. In parallel, Reference Architectures (RA) have been used to aggregate knowledge and architectural artifacts, capturing the systems' essence in specific domains. Therefore, it can be said that this type of architecture is an important way to support the development, standardization, and evolution of software systems. Considering this context, the main contribution of this paper is to present the second release of a reference architecture called RA4SaS (Reference Architecture for SaS). This architecture is based on reflection, a controlled adaptation approach, and a set of automated processes that support the development of SaS in both design and runtime. To show the applicability of our RA, we conducted a case study that explored three adaptation scenarios. As a result, we observe our RA has good potential to efficiently contribute to the SaS domain.

INDEX TERMS Reference architecture, self-adaptive software, reflection, runtime, automated processes.

I. INTRODUCTION

Software systems have played an important role in our society and operated in several segments, such as public and private institutions, airports, communication systems, among others. In parallel, the complexity of software systems and their computational environments has increased in the last few years. Thus, we must prepare these systems to operate under normal and adverse conditions and maintain their integrity of execution 24/7 (i.e., 24 hours per day and seven days per week). Therefore, features, such as robustness, reliability,

scalability, and flexibility have been increasingly required by these systems in the presence of changes (e.g., users' new needs and/or modifications in the execution environment). Such features match with a specific software system known as self-adaptive systems. Although this term has been employed in various domains, this article will focus exclusively on the software domain, referencing it from this point forward as Self-adaptive Software (SaS). This type of software enables the incorporation of some new features or behaviors at runtime without human intervention [1], [2], [3], [4], [5].

Software adaptation can become an onerous activity regarding time, effort, and money when manually performed. The human ability to manage systems has shown as

The associate editor coordinating the review of this manuscript and approving it for publication was Fabrizio Messina.

inadequate when their complexity increases. We have often observed the involuntary injection of errors/uncertainties as one of the major causes of system failures, especially in the SaS domain. To overcome these adversities, automated approaches have been adopted, since they are a feasible alternative to maximize the quickness of SaS implementation and minimize developers' involvement. In SaS, we move the design decisions towards runtime to control dynamic behavior and individual reasons for such systems about their states and environments [1], [2], [6], [7], [10].

According to Salehie and Tahvildari [1], Macías-Escrivá et al. [8], Passini and Affonso [9], and Affonso et al. [10], software in an execution environment imposes a set of challenges to perform structural and behavioral modifications without affecting its execution states. Requests for such modifications may be completely unknown to the software domain and they can occur for different reasons, such as [11]: (i) environment or context variability, when the environmental conditions have not been fully foreseen during development; or (ii) architectural or structural variability, when there is variability in the architectural components and/or the connectivity of such components can be dynamically reorganized or adapted.

From another perspective, Reference Architectures (RA) refer to a special type of software architecture that has become an important element to systematically reuse architectural knowledge [12]. Thus, different domains of software systems, including SaS, have understood the need for encapsulating knowledge (i.e., experiences and good practices) to disseminate and reuse this knowledge in the development of systems. For instance, Liu et al. [13] designed a reference architecture for the domains of service-oriented computing based on self-organizing mechanisms. Weyns et al. [14] developed a Reference Model (RM)¹ entitled FOrmal Reference Model for Self-adaptation (FORMS), which aims to support the development of systems based on formal methods and reflection. Affonso and Nakagawa [15] designed an RA for Self-adaptive Software called RA4SaS, which enables the adaptation of software entities² at runtime through an automated process and a controlled adaptation approach. Next, based on previous experience related to RA4SaS, we designed two RAs, namely: RA4Self-MobApps (RA for Self-adaptive Service-oriented Mobile Applications) [10]; and RA4Self-CPS (Reference Architecture for Self-adaptive Cyber-Physical Systems) [16]. Thus, it is noted that our research group has been involved with the design of reference architectures in recent years for three software systems domains. The design of such architectures has revealed the existence of an important intersection of knowledge regarding the adaptation of the software to be adapted

(i.e., software entities, Web services, and Cyber-Physical Systems).

Despite the use of RA has assumed a relevant role in the development of software systems, a study conducted by our research group in 2014 [17] and updated in this article [18], has revealed that although the use of RA has a relevant role in the development of software systems, SaS has been developed with no concern on the reuse of previously gained knowledge and experiences, such as best practices, guidelines, and better architectural styles. The main evidence of these studies shows us that a wide range of RAs and reference models was designed to meet specific domains of adaptation (e.g., autonomic control, service adaptation, and software components). The main goal of this article is to present the update of RA4SaS, an RA to support the development of SaS that was designed based on ProSA-RA (Process based on Software Architecture – Reference Architecture) [19]. In this second release of RA4SaS, our architecture contemplates two phases of development, namely: (i) entity design based on a controlled adaptation approach supported by a tool called DSLModeler4SaS; and (ii) runtime, which uses an automated process to adapt a software entity at runtime without the perception of its stakeholders. Therefore, it can be said that our architecture aims to consolidate itself to facilitate the development of SaS. In short, we can summarize the main contributions of this new release in four items:

- **Development of software entities through a text-to-text approach.** To do so, we have developed a Domain-Specific Language (DSL) that enables the software engineer to specify the software entities in a platform-independent language;
- **Development of a tool called DSLModeler4SaS.** We have developed a tool that enables modeling software entities using the approach mentioned in the first item. This tool has analyzers that enable the software engineers to verify that the encoding of the entities is written correctly, as well as the annotations inserted in such entities to model the interests of adaptation and persistence are right;
- **Automatic code generation.** We have coupled a source code generator developed in previous works by our research group to the DSLModeler4SaS tool to generate the source code of a SaS based on a set of templates [20], which contain design decisions and architectural patterns for SaS development. Therefore, it can be said that the result can boost SaS engineering activities because software engineers can focus their efforts on the design of entities and minimize their efforts on repetitive tasks. Indirectly, this activity can cause significant gains in development time, since the generated code is fault free (i.e., previously tested), minimizing the efforts directed toward the testing activity; and
- **Persistence annotation support.** We have developed a module that enables persistence annotations to be inserted into each software entity in DSL format. The annotations of this module follow the specifications

¹Reference model is a domain abstraction (i.e., a division of functionality together with data flow between the pieces) used in the design of a reference architecture [12].

²In our architecture, SaS is a program unit based on object-oriented programming concepts that can be referred to as a self-adaptive software entity or simply an entity.

of the existing persistence frameworks available in the literature (i.e., JPA – Java Persistence API³). Thus, developers can preserve their knowledge, since they are developing software entities (SaS) on a stack of technologies that they are used to developing.

The remainder of this article is organized as follows: Section II presents the background and related work; Section III describes the releases of the RA4SaS; Section IV presents a case study to show the applicability of our new architecture; Section V reports the discussion of results and limitations of our study; and Section VI summarizes our conclusions and perspectives for further research.

II. BACKGROUND AND RELATED WORK

This section presents the background (i.e., concepts and definitions) and related work that contributed to the development and evolution of our architecture. Initially, we report concepts of adaptation and self-adaptation software, computational reflection, and reference architecture. Next, we address related work on reference models and RAs for SaS.

A. ADAPTATION AND SELF-ADAPTATION

According to Villegas et al. [21], we can find different publications in software engineering and research communities that deal with the terms adaptation and self-adaptation as synonyms. Although such terms may have a small threshold, we understand that there are important differences and we believe that understanding the difference between both is a key aspect to design software systems that have to adapt at runtime. In short, adaptation software occurs at (re)design time and is manually performed by a software engineer in charge of maintenance tasks (e.g., software evolution). In self-adaptation, the software must be able to perform the adaptation at runtime with no or minimal human intervention.

According to Merriam-Webster⁴ and Cambridge Dictionary,⁵ some definitions of adaptation can be found: (i) “something that is adapted”; (ii) “the act or process of adapting”; (iii) “adjustment to environmental conditions”; and (iv) “the act or process of changing something to fit a new use or situation”. From an engineering view, “software adaptation has been defined as the process of changing the system to accommodate changes in its environment”. This definition does not provide evidence regarding the time of this activity, i.e., it is not mentioned whether the adaptation must be dynamic or must occur at runtime.

As argued by Laddaga [22] and Villegas et al. [21], software (or any piece of code) can be considered adaptive when it is easy to change, which is an activity that humans (i.e., developers) must conduct. A modification, regardless of its scale, requires changes to the source code, recompilation, and momentary interruption of the system execution. Self-adaptive software can evaluate its internal state and/or

execution environment and propose modifications without interrupting its execution. These modifications can occur when the functional and/or non-functional requirements of this software type are no longer meeting. We can convert adaptive software into self-adaptive if enabled with self-awareness features.

Making a SaS community compilation, this type of software has specific features compared to a traditional one because it enables structural, behavioral, or contextual changes at runtime. Among these changes, some of them address complexity management, robustness in handling unexpected conditions (e.g., failure), and change of priorities and policies governing the goals and conditions of its execution environment. According to Salehie & Tahvildari [1], “self-adaptive software is expected to fulfill its requirements at runtime in response to changes. To achieve this goal, software should have certain characteristics, known as self-* properties. These properties provide some variability, and help to overcome deviations from expected goals (e.g., reliability)”. To manage the changes at runtime, the feedback loop proposed by IBM [23] has been a feasible alternative because it takes all decisions based on a plan established in the data collected from the execution environment.

B. COMPUTATIONAL REFLECTION

Computational reflection, or simply reflection, can be defined as any activity performed by a system on itself, being like human reflection. The principal goal of software systems that present reflection is to get information about their own activities, aiming at improving their performance, introducing new capabilities, or even solving their problems by choosing the best procedure. Therefore, the use of reflection makes it possible for the software to be more flexible and more susceptible to changes [24], [25], [26]. Following, some primary uses of reflection for software adaptation are detailed.

A technique based on reflection for software components adaptation (i.e., structure and behavior) was proposed by Borde et al. [6]. The components are wrapped so that the existing functionalities are preserved and the new requirements can be added, constituting a new component. This technique also provides control over the number of modifications that can apply to software components, since they can quickly increase in size and their reusability could be reduced.

According to Pukall et al. [27] and Aßmann et al. [28], the reflection has been used successfully in the software components reuse and it has been implemented in the reuse of software architecture and its components. Both authors divided the architecture into two levels: (i) meta-level, which contains the architectural components, i.e., information describing the base-level as architecture topology, components, and connectors; and (ii) base-level, which can be traditional software architecture. Thus, it is observed that

³<https://www.oracle.com/java/technologies/persistence-jsp.html>

⁴<https://www.merriam-webster.com>

⁵<https://dictionary.cambridge.org>

there are already important initiatives for using reflection in the development of software systems.

C. REFERENCE ARCHITECTURE

In the study of Nakagawa et al. [29], we can find some essential definitions of RA. As one of the first definitions, Kruchten [30] stated that a “reference architecture is, in essence, a pre-defined architectural pattern, or set of patterns, possibly partially or completely instantiated, designed and proven for use in particular business and technical contexts, together with supporting artifacts to enable their use. Often, these artifacts are harvested from previous projects”. Next, in Bass et al. [12], RA was defined as “a reference model mapped onto software elements (that cooperatively implement the functionality defined in the reference model) and the data flows between them”. In this direction, Nakagawa et al. [29] defined that a “reference model can be considered as an abstract framework that presents a minimal set of unifying concepts, axioms, and relationships within a particular problem domain, independent of specific standards, technologies, implementations, or other concrete details.” In the view of Rosen et al. [31], an “reference architecture is a working example of a critical aspect of your enterprise architecture that enables to work with your organization’s message bus or business rules engine”. In parallel, Angelov et al. [32], [33] stated that an “reference architecture is a generic architecture for a class of information systems that is used as a foundation for the design of concrete architectures from this class”. Finally, Reed⁶ stated that an “reference architecture consists of information accessible to all project team members that provides a consistent set of architectural best practices”.

Because of the aforementioned definitions, Nakagawa et al. [29] stated that a “reference architecture refers to an architecture that encompasses the knowledge about how to design concrete architectures of systems of an application domain; therefore, it must address the business rules, architectural styles (sometimes also defined as architectural patterns that address quality attributes in the reference architecture), best practices of software development (for instance, architectural decisions, domain constraints, legislation, and standards), and the software elements that support the development of systems for that domain. All of this must be supported by a unified, unambiguous, and widely understood domain terminology”. In parallel, Angelov et al. [32], [33] reported that we can use architectural patterns and reference models in the design of RAs, as illustrated in Fig. 1.

Although the definitions reported in this section provided us with different views on RA, the reuse of knowledge about software development in a domain regarding architectural design is common sense among the authors. Evidently, we can find other definitions of RA in the literature, but we

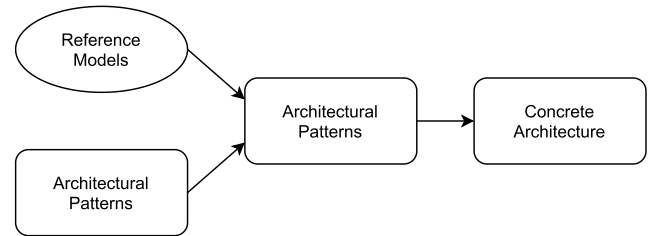


FIGURE 1. The relationship among reference models, architectural patterns, RAs, and concrete architectures (adapted from Angelov et al. [32], [33], arrows indicate model data input).

believe that the content reported in this section suffices to contextualize the work proposed in this article.

III. RA4SaS: REFERENCE ARCHITECTURE FOR SaS

This section presents an overview of the RA4SaS. Initially, a description of the first release is provided in Section III-A. Next, a description of the second release is presented in Section III-B, highlighting the novel contributions of this architecture to support the development of SaS.

A. FIRST RELEASE

As mentioned in Section I, we used the ProSA-RA, a process to build RAs proposed by Nakagawa et al. [19] to create the first release of RA4SaS. In short, this architecture is based on reflection [24], [42], an important resource for inspection and modification of software entities at runtime. In the first release, this architecture provided its stakeholders with a set of guidelines that aims to support the development of the software entities through a controlled adaptation approach. To do so, software engineers must define the adaptation level of each software entity in the development phase. This architecture also provides an automated process that enables the monitoring and adaptation of these entities in the execution environment at runtime based on adaptation levels defined in the design phase without human intervention. Fig. 2 shows the architecture is composed of an adaptation core (dotted line) and four complementary modules: Development, Action Plan, Adaptation Rules, and Infrastructure.

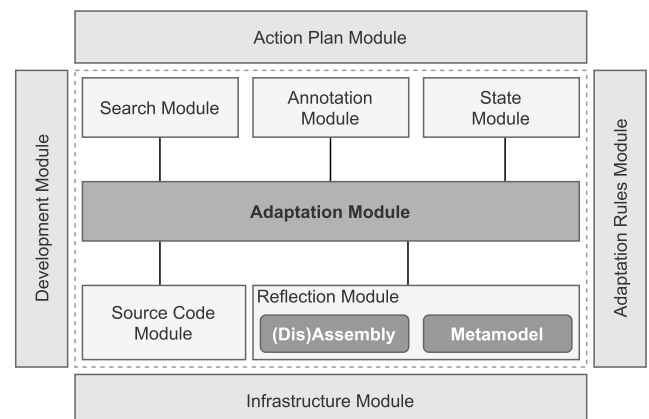


FIGURE 2. RA4SaS: first release [15].

⁶<https://blog.naver.com/inforide/80020691162>

According to Affonso and Nakagawa [15], the RA4SaS has an automatic process for SaS adaptation that represents a logical step sequence (i.e., module orchestration) so that a software entity can be adapted. In a closer examination, it acts as a supervisor system of the software entities, monitoring their requisitions in the execution environment. In summary, an entity from this environment is disassembled, and a metamodel is instantiated. Next, this metamodel is adapted based on new information (i.e., structural and/or behavioral interests), and automatically reassembled as an “assembly line”. It is important to emphasize that this process must be performed by automated processes, which reduces the implementation complexity and minimizes the involuntary generation of uncertainties.

Finally, as this article aims to present the evolution of RA4SaS as a second release, we do not report in this section the details concerning architecture modules and its automated process to overcome the redundancy of information. Therefore, the updated description of these modules and the new ones, besides the automated process, will be presented in Section III-B.

B. SECOND RELEASE

Before presenting the second release of the RA4SaS, it is important to highlight that the design of the new release was also guided by Prosa-RA. In a previous study of our research group, we investigated the information sources and reported some evidence regarding the relationship between the representation of the dynamic behavior and the type of knowledge used in the design of RAs for SaS [17]. Next, as a way of tracking the evolution of this research topic, we conducted a Systematic Mapping Study (SMS) [34], which will not be reported in this article for space reasons. Details of the research protocol used in the aforementioned SMS can be viewed in Affonso et al. [18]. Therefore, information extracted from this mapping combined with the systematic review published by the research group, formed the knowledge basis for the design of our new architecture. Based on this knowledge source, we have defined a new set of architectural requirements for the RA4SaS, which is composed of the first version [15] and new requirements related to the evolution of our architecture. These requirements will not be presented in this article for scope and space reasons, but a new list can be viewed in Affonso et al. [35].

Besides patterns and RAs used to establish the final list of requirements, we also verified before other information sources, namely: (i) development of SaS and its engine of adaptation [1], [2], [3], [4], [7]; (ii) SaS developers with experience in designing and implementing SaS [5], [10], [14], [16], and (iii) examples of systems [36]. Regarding the last item, it is noteworthy that the aforementioned authors developed such systems for different domains and they provided excellent contributions (e.g., adaptation scenarios, architectural requirements, end-user needs, among other features) to the design of our architecture.

Regarding the architectural design of RA4SaS, we address a panorama that encompasses the goals and stakeholders for our architecture. Concerning the **goals**, we designed our architecture to meet a feature set, namely: (i) it supports the development of SaS; (ii) it contributes to better modularization, structuring, and maintaining the developed systems (SaS); (iii) it contributes to better reuse of software elements developed based on SaS; and (iv) it intends to support the SaS evolution (i.e., simple and composite entities). As **stakeholders** in SaS, we can mention the following actors: (a) software architects, responsible for identifying system needs and deriving the concrete architecture; (b) domain specialists, responsible for providing specific information and verifying domain requirements; (c) analysts, responsible for validating the set of information in the concrete architecture; (d) quality assurance managers, responsible for ensuring the achievement of requirements in systems; (e) developers, responsible for instantiating a reference architecture in a concrete architecture; (f) maintainers, responsible for maintaining and updating such systems; and (g) end-users, responsible for interacting with the system. Although we have presented these actors as possible stakeholders in RAs for SaS, it is important to highlight that a reference architecture does not encapsulate all potential actors/stakeholders; thus, the mentioned ones are the most relevant. Next, we address details regarding the design of our reference architecture.

Fig. 3 illustrates a second release of the RA4SaS. Regarding the design differences between the two releases, the following modifications should be highlighted: (i) removing the adaptation rules module because of its loss of functionality in the architecture. (ii) the **Annotation Module** was renamed to **Entity Annotation** for nomenclature reasons; (iii) addition of the **Persistence Annotation** and **Logging** modules to the **core** of our architecture (dotted line) to meet the new requirements; and (iv) addition of the **DSLModeler4SaS Module** as a modeling tool for software entities within our architecture. These last two modifications (i.e., items iii and iv) were highlighted by boxes in solid black lines, with the symbol “N” to denote a new module. Concerning the operational mode, the **Entity Annotation Module** lost its functionality in the design phase of software entities for DSLModeler4SaS tool and a DSL named eLanguage (see Section III-B6). In the new release of RA4SaS, this module is used by the **Reflection Module** to manage annotations when an entity is disassembled and a metamodel is instantiated. The remaining modules from the first release had their functionality preserved and/or evolved in the second release. Next, we present a description of each module in an execution logical sequence.

1) DEVELOPMENT MODULE

The SaS development has specific features compared to the traditional one since this type of software enables changes (i.e., structural, behavioral, contextual, or mixed of both) to be incorporated at runtime. It also requires different approaches, an automatic process for adaptation, and flexible

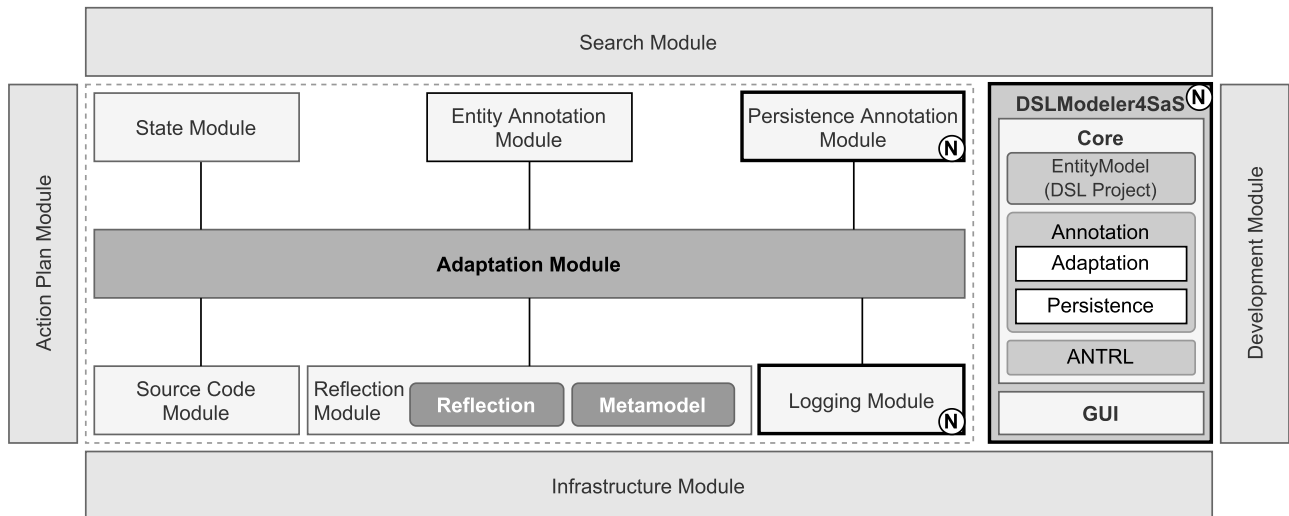


FIGURE 3. RA4SaS: second release.

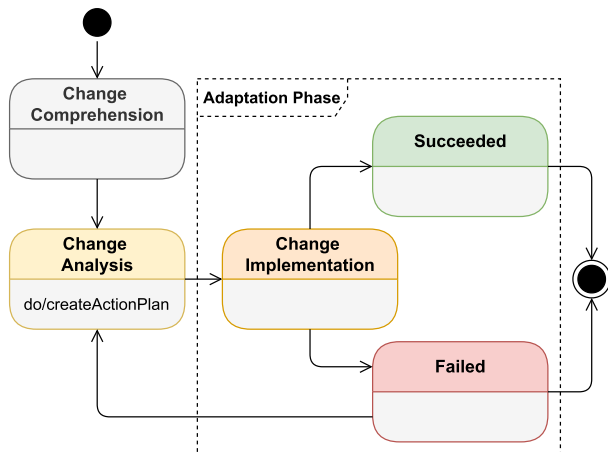


FIGURE 4. Requirement states in the adaptation process [37].

architectural models. Thus, this module provides a set of guidelines for the development of SaS (i.e., software entities) based on a controlled adaptation approach and a non-intrusive strategy. These guidelines act in the following phases: requirements analysis, design, implementation, and evolution of such entities at runtime. Initially, software engineers and domain specialists must comprehend the problem based on requirement specification so that they can assess the development viability. Supported by the **Search Module**, they search in the repositories of software entities to find entities of identical or similar features. As a result, they can reuse identical entities and dispense a development approach, or adapt an existing one. Similar entities undergo an evaluation process so that changes (i.e., requirements and goals) to be implemented can be validated. The states of off-line adaptation of the software entities are illustrated in Fig. 4.

Preliminary, it can perform a comprehension of changes so that analysis regarding the viability of these changes

(Change Comprehension and Change Analysis states). Next, the adaptation phase of this process is conducted (dotted line) so that the changes can be applied. In short, when a change activity is being performed Change Implementation state, two states can be assumed: (i) **Succeeded**: when a change has been successfully implemented and the activity is completed (**final state**), or (ii) **Failed**: when a failure has been detected. At this moment two transitions can be performed: (1) return to the analysis step for the reevaluation of the action plan and start of a new trial of adaptation, or (2) termination of the activity (**final state**), i.e., the design phase can be started [37].

Regarding the design, we can develop software entities based on an object-oriented paradigm. After the entity conception, developers must insert annotations to define the adaptation level of each entity, characterizing a controlled adaptation approach (see Section III-B6). Our architecture also enables the business logic of these entities can be organized in different layers, depending on the templates of source code that will generate the target system (see **Source Code Module**). In this new release, our architecture enables us to deal with entities that require information storage in the database too (see **Persistence Annotation Module**). According to Hibernate [38], the development of object-oriented software with relational databases can be cumbersome and time-consuming due to a number of “paradigm mismatches” between data types (i.e., objects versus relational databases). To overcome these adversities, developers can insert annotations to define the relationship between entities that require data storage in the database as an Object-Relational Mapping (ORM) solution. For instance, when developers create software entities, the **Source Code Module** can generate the persistence layer based on templates elaborated during the design phase. Next, when these entities are adapted at runtime, the **Source Code Module** can generate the persistence layer to meet new features

(i.e., functional or non-functional requirements) added or removed.

We can consider the evolution of the adaptive system as a special issue because it aims to capture changes in a software entity during its life cycle. Initially, when a software entity is developed, its goals are defined in parallel too. Thus, when this entity is adapted at runtime or design time, its goals can be also modified. This scenario reveals that disordered adaptations can cause the entity to increase in size quickly and could make their use in future systems not feasible. Therefore, to establish management on the adaptation number of these entities and their goals, we opted to use a base of metrics that enables evaluating the complexity and behavior of entities in both the design and runtime phases (see **Rules Adaptation Module**). Such metrics aim to control the cohesion and granularity of entities so that they do not lose their flexibility and adaptation capacity.

Finally, it is noteworthy that the software entities operate in a supervisor-supervised model based on reflection. In short, this model represents a non-intrusive adaptation approach with a segmentation between the adaptation logic (supervisor system – meta-level) and the target system (supervised system – base-level). Regarding operations, four steps summarize the adaptation process of the software entities at runtime, namely: (i) interception of entity supervised, (ii) analysis of adaptation feasibility, (iii) establishment of an action plan for adapting, and (iv) implementation of changes in the software entity.

2) SEARCH MODULE

This module aims to assist in the manual and automatic search for entities in the repository in both phases: the development of a new software entity and/or adaptation of an entity at runtime. To support such phases, this module provides two search methods: (i) semantic, which can be defined as a search query through contextual meaning for software entities. The software engineers/developers insert the semantic information during the development phase to describe the structure and behavior of each software entity. This type of search provides more meaningful results by finding the most relevant entity in the repository; (ii) technical, which can be specified by only entity information (i.e., Entity, Attribute, or Method) in template format, which is converted as input parameters to search in the entity repository for some type of matching. In a manual search, the software engineers and/or developers decide on the similarity of the entity by analyzing the returned data. In the automatic search, we can combine such methods for selecting an entity.

3) ACTION PLAN MODULE

This module aims to assist in the adaptation activity of software entities by providing means to control dynamic behavior, individual reasons, and execution states of each software entity concerning the new changes (e.g., user requirements or execution environment). The combination of the answers from the 5W1H model [1] to adapt a software

entity and MAPE-K (Monitor, Analyze, Plan, Execute over Knowledge base) control loop proposed by [23] to support the decision-making has emerged as a feasible solution to an action plan for SaS community [36] and our architecture [15].

Regarding the relationships between the 5W1H model and the MAPE-K control loop, the evidence shows that the questions of the aforementioned model are spread in all phases of the loop. The Monitor process aims to collect data from sensors for converting them into behavioral patterns and symptoms. We relate this process to “Where”, “When”, and “What” questions of the 5W1H model. The Analyze process aims to correlate the data from the Monitor so that complex situations can be modeled so that decision-making systems can learn from the environment and predict future situations. This process helps to identify “When” and “Where” the modifications must be applied. The Plan process is responsible for creating “What” will be adapted and “How” to apply the changes in the software entity to achieve the best results. The Execute process must provide the mechanisms that can execute the established action plan. To do so, this process relates to the questions of “How”, “What”, and “When” to change. Finally, the monitors “Sensors” and “Effectors” are software entities that can generate a collection of data reflecting the state of the system and rely on in vivo mechanisms or autonomous subsystems to apply such changes [1], [23].

Based on the exposed scenario, our research group developed a framework for decision-making for the SaS domain [39]. The main purpose of this framework is to classify and analyze sensory data to autonomously detect and mitigate failures at runtime. To do so, this framework uses learning techniques and the MAPE-K control loop. Regarding the structure, we organize this framework into two modules: classification and recommendation. The first one aims to present a classification for a data set collected via sensors from the execution environment. The second one aims to present a solution set ranked by statistical measures for a problem reported by the classification module. To automate the development of these modules, our research group developed a tool called DmS-Modeler, providing facilities for modeling, calibration of such modules, and automatic generation of infrastructure [40]. For scope reasons, details of this framework and DmS-Modeler tool are not presented in this article but can be obtained via the aforementioned references.

4) INFRASTRUCTURE MODULE

This module aims to support the software entities’ adaptation process at runtime. In summary, an entity can be deployed, undeployed, and redeployed at runtime without the application/system being interrupted. To do so, this module has three mechanisms, namely: (i) dynamic compiling and dynamic loading, which enables the replacement of a software entity at runtime without restarting the application or redeploying its components; (ii) diagnosis of problems, which enables

the identification and reporting problems occurring when an adaptation activity is being performed; and (iii) self-healing, which enables fixing problems/failures at runtime when an adaptation activity is being performed.

According to Salehie and Tahvildari [1], Lemos et al. [3], Krupitzer et al. [7], and Affonso et al. [10], there are two approaches for the adaptation of software entities at runtime: (i) internal, characterized by the intertwining between SaS and the adaptation logic; and (ii) external, characterized by the separation between the SaS and the external adaptation mechanism (or manager), which contains adaptation processes. As mentioned in Section III-B1, our architecture uses a non-intrusive adaptation strategy with an external logical adaptation. We designed the aforementioned mechanisms in an external module to monitor the software entities in the execution environment and adapt them when new adaptation interests were detected.

5) ADAPTATION CORE

The structure indicated by the dotted line must be considered the “heart” of our architecture because it enables managing software entity adaptation at runtime. To do so, this structure is composed of seven modules: (i) Entity Annotation; (ii) Persistence Annotation; (iii) Reflection; (iv) Source Code; (v) State; (vi) Logging, and (vii) Adaptation. Regarding the functionality of these modules, it is worth highlighting the last one is the most complex of all since it represents a logical sequence of well-defined steps (i.e., “orchestrator”) so that the adaptation of the software entities is conducted through an automated process as an “assembly line”. Next, we report a description of each module, highlighting the main functionalities, models, and source code (as a running example) to show the RA4SaS’s operating logic. Regarding the source code presented in this section as a running example, Java was chosen because it is the same programming language used to instantiate RA4SaS in a concrete architecture, as shown in Section IV.

Entity Annotation Module: We developed this module to assist the software engineer in defining the adaptation level of software entities in the design phase (i.e., first release). As our architecture works with a controlled adaptation approach, this metadata (i.e., annotation) shows the adaptation level supported by the software entities. In short, the **Reflection Module** uses this module to identify what information can be modified (i.e., removed and/or added) during the runtime phase. In terms of operations, this module enables verification if the developers inserted the annotations in each entity during the design phase. Moreover, this module also enables the recovery of annotations inserted in software entities during the design phase at runtime so that the **Reflection Module** can achieve the modifications in each entity.

In Listing 1, examples of adaptation annotations for an information system for managing a car rental are addressed. It is worth highlighting that the adaptation annotations supported by RA4SaS can be inserted both

```

1  @ClassAnnotation(className = "Car",
    ↪ adaptationLevel = CLASSES,
    ↪ semanticInformation = "...", type =
    ↪ BUSINESSCLASS)
2  public class Car {
3
4      @FieldAnnotation(attributeName =
    ↪ "licensePlate", semanticInformation
    ↪ = "...", primaryKey = "no")
5      private String name;
6
7      private String licensePlate;
8
9      public Car() {
10     }
11
12     @MethodAnnotation(methodName = "validate",
    ↪ semanticInformation = "...",
    ↪ parametersName = "", adaptable =
    ↪ Adaptable.YES)
13     public void validate() {
14         //...
15     }
16 }

```

LISTING 1. Entity annotation examples.

in the source code of the target programming language, as exemplified in the aforementioned listing, and in the DSL code (see Section III-B6). Regarding the annotations, the `Car` class in Java code was annotated (`@ClassAnnotation` – line 1) to support adaptation at the class level (`Artifact.CLASSES`). This value indicates to the automated mechanism of our architecture that this entity can support structural and behavioral changes. Thus, we must insert annotations to define which attributes (i.e., `FieldAnnotation` annotation – line 6) and methods (i.e., `MethodAnnotation` annotation – line 12) can be modified at runtime.

Regarding the remainder adaptation levels, when an entity supports only structural modifications, the `adaptationLevel` field must receive `FIELDS` value so that it can change only the attributes of this entity at runtime (i.e., added or removed). To do so, we must insert a `FieldAnnotation` in each attribute of this entity that can be changed. Similar to attributes, when an entity supports only behavioral modifications, the `adaptationLevel` field must receive `METHODS` value so that it can change only the annotated methods at runtime. Thus, we must insert a `MethodAnnotation` in each method of this entity that can be changed.

Persistence Annotation Module: We designed this module to assist the software engineer in developing software entities that need to persist information in the database. Data persistence is a means for an application to persist and retrieve information from a nonvolatile storage system (e.g.,

database, file system). As this activity can be costly in terms of development, we designed a **Persistence Annotation Module** based on ORM persistence frameworks supported by JPA specification with the following features: (i) common specification that enables handling data persistence; (ii) object-relational mapping through annotations; and (iii) implementations for several target programming languages. We choose this type of framework by the automation they offer to generate data schemas and to provide operations for data persistence. This type of framework minimizes the developer's effort during the development phase and automatic code generation processes of our architecture (RA4SaS) at the runtime phase.

Regarding use, software engineers must annotate the object-relational mapping relationships in each entity, following the specification patterns of this type of framework. To do so, this module has a metamodel to represent the persistence annotations of an entity similar to the metamodel of software entities (see **Reflection Module**). Because of the similarity and low complexity of this module's metamodel, its design details will not be reported in this article, only the metamodel of the **Reflection Module** will be detailed. Concerning the operation mode, when the **Reflection Module** “disassembles” an entity during the adaptation process, the information about this entity, including persistence annotations, is retrieved and inserted into this metamodel to meet new requirements or adaptation needs. Details of the adaptation process will be presented in the **Adaptation Module**.

Reflection Module: We designed this module to assist the software engineer in adapting software entities at runtime, as shown in Fig. 5. To do so, this module has a metamodel⁷ that represents a software entity and a set of operations that enables the “disassembly” and “assembly” of this entity so that an adaptation activity can be achieved. Regarding entity disassembling, the automated process of our architecture must use the entity annotations (attributes in blue) and persistence annotations (attributes in red) modules to get the adaptation level supported by an entity, as well as its persistence information. Next, the disassembly process can be started so that structural and behavioral information (attributes and methods) can be recovered via reflection and inserted into a metamodel. After instantiating this metamodel, new information according to adaptation interests can be inserted into or removed from this new metamodel so that it can generate a new entity. To do so, this new metamodel must be transferred to the **Source Code Module** to create the new software entity.

As illustrated in Fig. 5, the metamodel enables the adaptation of object-oriented software systems or makes use of the structure of classes as units of software systems. This strategy can be relevant for developers because they can develop adaptive software entities in a notation close

to their expertise area. Regarding the initialization of the metamodels, `Project` class is the starting point because it receives a list of objects (`List <Object>`), which represents all software entities that must be adapted. These entities are the variables of software systems (e.g., objects, software components, and web services) existing in a software project. The disassembly process retrieves structural and/or behavioral information on software entities and inserts it into a metamodel. To do so, the Reflection API⁸ is used as a computational resource to find information at runtime [24], [42]. Although this metamodel has reflective features, it represents a generic solution. Thus, other computational resources (i.e., programming languages) can also be used to instantiate it, since they meet the pre-requirements mentioned in this section.

Finally, it is worth mentioning that the **Reflection Module** must implement operations to manipulate the metamodel and the software entities, namely: (i) information retrieval from a software entity at runtime, (ii) instance of the metamodel with the information got in Step i; and (iii) transference of this metamodel (Step ii) to the **Source Code Module** so that it can generate the source code of each new software entity. For space and scope reasons, details of these operations will not be reported in this article.

Source Code Module: This module aims to generate the source code of the new software entities (i.e., those adapted at runtime) based on the metamodel instantiated by the **Reflection Module**. This module must be designed based on a template engine, which enables for generation of text output based on models and variable data. In general, template engine frameworks use a simple and specialized language to write a system's templates, which differs from programming languages (e.g., Java). Based on this context, the software engineer must provide a set of software entity templates based on the architectural pattern (e.g., logical layer, persistence layer, and others) for each target system so that the runtime adaptation be achieved.

Concerning operational mode, this module should have three functionalities to generate the source code that meets the adaptation interests: (i) structural, when only one or a list of attributes should be added or removed from an entity. Here, specifically, the `getters` and `setters` methods that manipulate these attributes are modified; (ii) behavioral, when only a list of methods should be added or removed from an entity; and (iii) structural and behavioral, when one or a list of attributes and one or a list of methods should be added or removed from an entity. Finally, it is noteworthy that this module must implement a mechanism for version control of the entities' source code, preventing them from being overlapped and keeping versions of all stakeholders.

State Module: This module aims at preserving the current execution state of the software entity. When an entity is selected for adaptation, the information contained in its current state must be preserved. At runtime, an entity can

⁷Logical information model that specifies the modeling elements used within another (or the same) modeling notation [41].

⁸<http://docs.oracle.com/javase/tutorial/reflect>

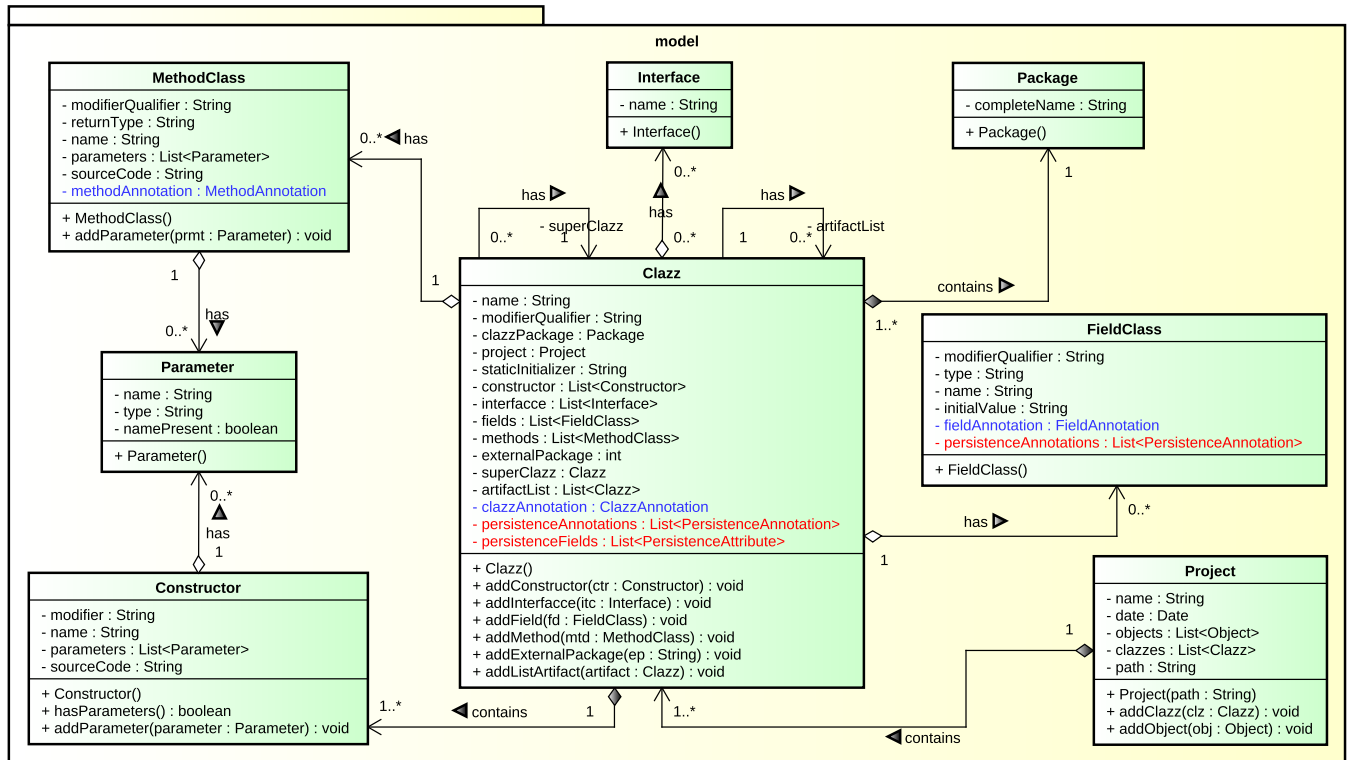


FIGURE 5. UML model for the metamodel of the reflection module.

be modified and the information must be recovered and reinserted so that the original execution state (i.e., current information) is not interrupted. To do so, this module should provide two functionalities, namely: (i) serialization, which enables the conversion of an entity into a file (.json) with structural information and its content, and (ii) deserialization, which represents the reverse process in relation to the first one (i.e., file to an entity). The choice of JSON⁹ (JavaScript Object Notation) to perform these operations is related to the following facilities: file handling (reading and writing), integration with different programming languages, and low cost of implementation.

Logging Module: This module aims to generate the log information of the adaptation operations in both the design and runtime phases. First, software engineers and developers must use a tool called DSLModeler4SaS, which was developed by our research group and will be presented in this article (see **DSLModeler4SaS Module**). When a software entity is designed, annotations that deal with the adaptation level (**Entity Annotation Module**) and persistence (**Persistence Annotations Module**) of the software entities must be inserted. As reported in the description of these modules, software engineers and developers must insert such annotations correctly so that the entities' source code can be generated for a target programming language. When an annotation has not been inserted or when it has

been inserted wrongly, the log information is captured and displayed in the output area of the DSLModeler4SaS tool. Second, software entities are adapted to meet the new interests of their client applications and/or context. In order to track the adaptation activities, log information must be collected to be further analyzed by the stakeholders (i.e., software engineers, developers, and automated processes) in case these activities result in a broken state for the software entity. We can synthesize the states of the software entities in their execution environment in four items, namely: (i) *normal*, which represents a system consistency (without failures); (ii) *degraded*, which represents some system instability; (iii) *broken*, which represents the occurrence of failures. Here, the system requires human intervention to return to the normal state of execution; and (iv) **evolution**, which represents changes applied to the system to address new user requirements or the execution environment's new needs. However, when the evolution operation is unsuccessful, the system may go into a broken state. As can be observed, the log operations are important to support the development and generation of source code in both the design and runtime phases.

Adaptation Module: We consider this module the “orchestrator” of our architecture since it performs calls and coordinates all activities of the other modules when a software entity is adapted at runtime. In short, an application based on our architecture must be organized in two layers, namely: supervised and supervisor. The first represents the

⁹<https://www.json.org>

target application (i.e., software entities) that can be modified at runtime. The second represents the adaptation engine of our architecture that enables monitoring the requests of the software entities in the execution environment. To do so, this layer implements an adaptation process organized in a well-defined step sequence so that a software entity is adapted, as illustrated in Fig. 6. Next, we address a description of these steps for the adaptation of an entity at runtime.

As presented in Fig. 6, we organized this process into nine steps (A to I). Preliminary, software engineers can develop a software entity based on guidelines provided by the **Development Module** of our architecture. Alternatively, they can search for a software entity in the entity repository so that they can evaluate the reuse possibility of this entity (see **Development Module**). Then, starting from a software entity, the adaptation level (see **Entity Annotation Module**) and persistence annotation (not mandatory) (see **Persistence Annotation Module**) of each software entity must be recovered (Step A) so that a metamodel can be instantiated (Step C). Step B aims to preserve the current execution state of the software entities during the adaptation process. This information can be reused in Step I if the software entity has not been modified in terms of application domains (goals). In Step C, software entities are “disassembled” and a metamodel must be instantiated by the **Reflection Module**. This metamodel has the structural and behavioral information of each software entity, besides its annotations got in Step A (i.e., adaptation level and persistence annotation). In Step D, an action plan for the adaptation of each entity must be established. This step is a macro activity (dotted line) that is composed of four steps (D, E, F, G). The action plan must establish adaptation criteria so that the entity’s goals do not become unfeasible, preventing future adaptations. For instance, an entity can be modified to meet the new interests of its customer, preserving its application domain and goals. From another perspective, an entity can be modified and become a new entity with changes in its application domain and/or goals. Based on the goals and interests of adaptation, the **Action Plan Module** analyzed the changes (Steps E). Next, supported by the guidelines of the 5W1H model, structural and behavioral information (Step F) is inserted into the metamodel obtained in Step C so that a new metamodel is generated (Step G). The metamodel of each software entity is transferred to the **Source Code Module** (Step H) so that it can generate the software entities. In Step I, each entity is compiled and inserted into the execution environment. Regarding the last step, we highlight two considerations: (i) state restoration, and (ii) compilation process. First, if the entity changes do not impact the domain change (goals), the preserved information (Step B) is reinserted into this new entity to replace the old one in the execution environment. The replacement process of the software entities is performed transparently in relation to their stakeholders because they have no perception of the changes in the execution status and the new instances generated. Second, is the ability to interpret the error

messages by the **Infrastructure Module** when a software entity is being dynamically compiled/loaded (see `compile`, a subpackage of the **Infrastructure Module**). This feature is very important for the compilation process because these messages are useful information for the corrections in the source code. Note that all the operations (Steps A to I) described are tracked by the **Logging Module** (see `logging` package – gray area) so that problems can be recognized by the automatic mechanisms of our architecture or by the human when occurs an interruption because of a system crash. Finally, it is essential to emphasize that software engineering tools must perform this process automatically, intending to reduce implementation complexity and minimize uncertainty generation, becoming this process as a feasible solution for software adaptation at runtime.

6) DSLModeler4SaS MODULE

As can be seen in Fig. 3, we have organized the DSLModeler4SaS tool into two submodules, namely: Core and GUI (Graphical User Interface). The **GUI Module** has the responsibility of generating the graphical interfaces of the tool. To do so, we used the Observer [43] design pattern, which made it easier and more flexible to notify the tool’s interfaces when an operation is performed on a project. Finally, it is worth mentioning that this strategy made it possible to reduce and modularize the project into smaller parts, making the tool more flexible for future updates. The **Core Module** aims to manage the projects generated by the developers in DSLModeler4SaS, besides providing interfaces for the auxiliary modules (e.g., **Source Code Module**). Concerning the design of software entities, this tool works on a text-to-text approach. In summary, developers can use a DSL language to model the software entities and, supported by DSLModeler4SaS, configure templates for code generation in a target programming language. Next, we address a description of each module.

GUI: This module represents the graphical interface of the DSLModeler4SaS tool built on top of the **Core Module**. For scope reasons, details about the design and implementation of the tool will not be reported in this article, we will highlight only the main functionalities. As illustrated in Fig. 7, we can describe the tool through six elements. The **top menu (1)** enables accessing all the tool’s features (for example, creating projects, editing texts, and settings, among others). Furthermore, it is worth noting that several items in this menu are associated with keyboard shortcuts, which quick up the use of the tool. The **shortcut bar (2)** has a set of buttons for quick access to the most common features, such as creating projects, entities, and packages, saving, and building projects. The **tree in (3)** displays the projects currently open in the tool and facilitates navigation between its elements (i.e., packages, entities, libraries, and templates). The **editor panel (4)** is the region where the entity editors and templates are presented. This panel enables multiple instances of an editor to be open, creating separate tabs for each file. The

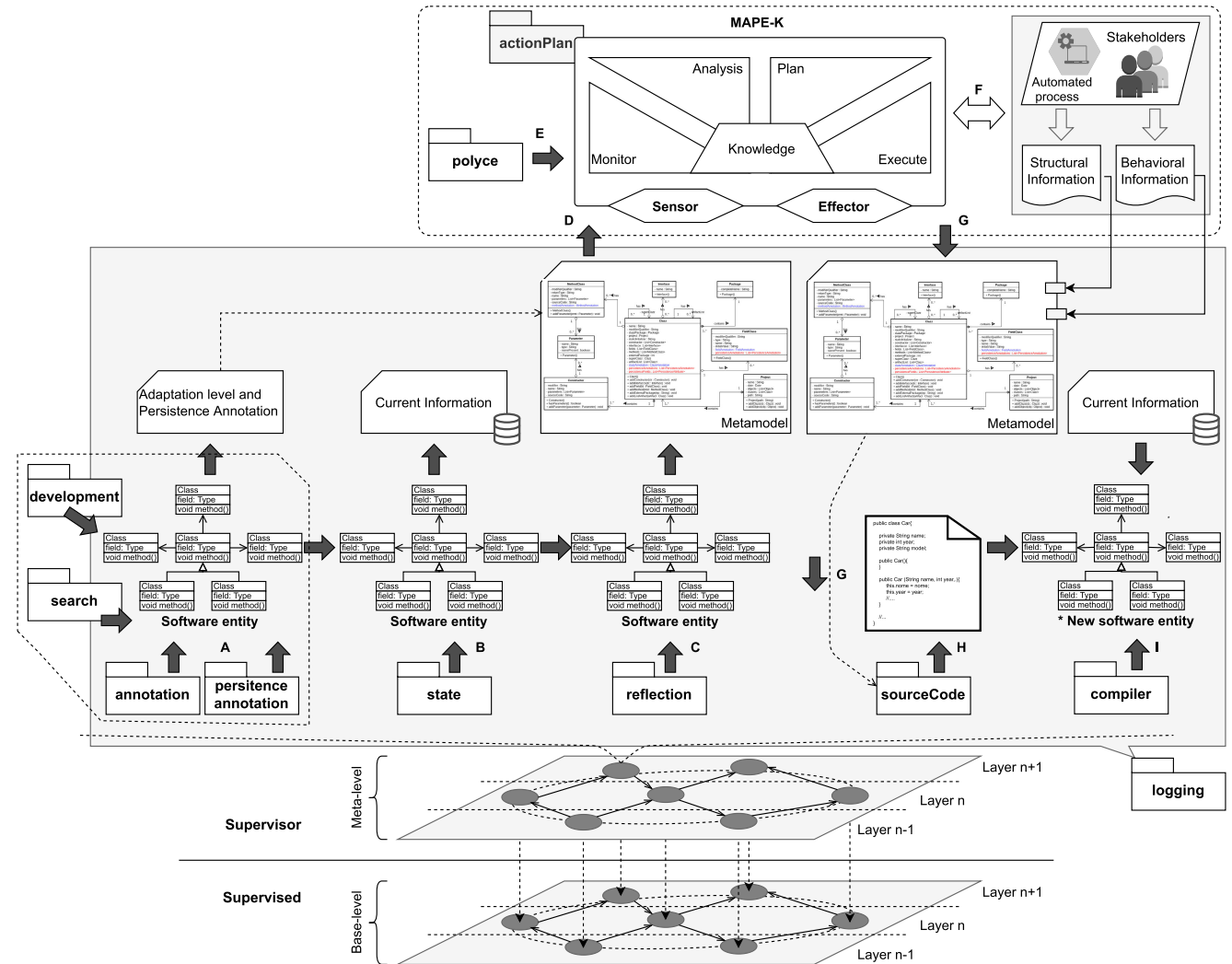


FIGURE 6. RA4SaS's adaptation process.

inspector area (5) enables us to view the attributes and methods of an entity selected in the project tree. Finally, the **log and error tabs (6)** were also implemented in the tool to present the user with messages about the activities performed by the developers (e.g., error information).

Core: We consider this module as the “background engine” of DSLModeler4SaS because it provides all the operations for the tool to work. As presented in Fig. 3, we organized this module in three packages: EntityModel, Annotation, and ANTLR. Next, we address a description of each package, highlighting the main models and essential functionalities related to the DSLModeler4SaS tool.

The **ANTLR package** contains the infrastructure necessary for the DSL written for the DSLModeler4SaS tool called eLanguage (i.e., entity **L**anguage). To facilitate the work of processing the language, we use the ANTLR tool (ANother Tool for Language Recognition) [44] because it enables us to generate interpreters for languages from the writing of grammar, avoiding the repetitive and error-prone work of

writing all the components of manual analysis. In other words, ANTLR tool also helps in the traversal activities of the generated Parse Tree and in the ways of issuing errors to the user interface (**GUI Module**). For space and scope reasons, details of the generation process and integration of our grammar eLanguage to the DSLModeler4SaS tool will not be reported in this article.

Besides DSL, the **ANTLR package** is composed of a class set that handles the lexical and syntactic analysis of software entities (SaS) in DSL format. Such classes enable extracting all the adaptation and persistence information in the source files of each entity for generating internal files (i.e., file project) of our tool and models (i.e., metamodel of the **Reflection Module**) in memory for generating source code by **Source Code Module**. Listing 2 shows the source code in DSL format for the design of an entity called *Customer*.

Analyzing the source code presented in Listing 2, some features of the eLanguage DSL can be observed. We declared four entities in this listening, namely: *Contact* (line 1),

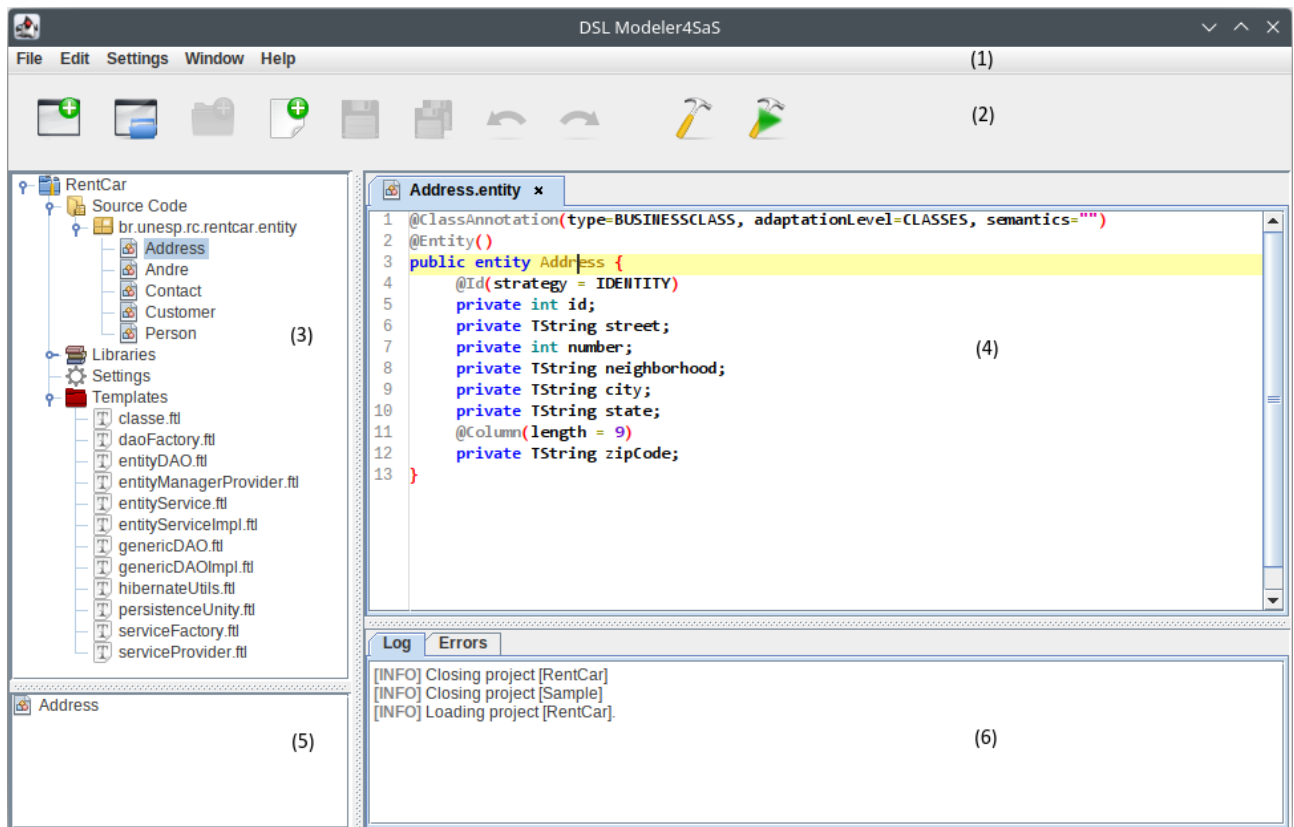


FIGURE 7. Main screen of the DSLModeler4SaS tool.

```

1 public entity Contact {
2     //...
3 }
4
5 public entity Address {
6     //...
7 }
8
9 public entity Person {
10     private int id;
11     private TString name;
12     private Contact contact;
13     private TList<Address> address;
14 }
15
16 public entity Customer extends Person{
17     private TString individualRegistration;
18 }

```

LISTING 2. Customer entity's source code.

Address (line 5), Person (line 9), and Customer (line 16). The attributes of the first two entities were omitted for scope and space reasons. On the other hand, in lines 12 and 13 it is possible to view the relationship attributes

of the Person entity to the Contact and Address entities, respectively. Finally, one can observe the use of the `int`, `TString`, and `TList` types in the Person entity, as examples of the use of types supported by the eLanguage DSL.

The **Annotation package** contains the annotation models (i.e., level of adaptation and persistence) that will be used during the elaboration of software entities in DSL format. Regarding these annotation models, it can be said that they operate similarly to the respective RA4SaS modules, but differ in terms of the code and modeling target. For instance, the target code is the programming language used to instantiate RA4SaS, while here the target is the eLanguage language. Moreover, this package provides some important features for modeling of a project in our tool, namely: (i) an infrastructure responsible for analyzing the source code of software entities in DSL format, besides verifying if developers inserted such annotations correctly; (ii) an interface for communication with the RA4SaS **Source Code Module** after the design and configuration of a project; and (iii) a set of classes for handling project files (i.e., entity files, project configuration, logs, among others) and other minor needs (utilities) of the tool. To show the use of the annotations in our DSL, we will extend the example in Listing 2 by adding adaptation and persistence annotations, as presented in Listing 3.

```

1  @ClassAnnotation(type=BUSINESSCLASS,
    ↪ adaptationLevel=CLASSES, semantics="")
2  @Entity()
3  public entity Contact {
4      //...
5  }
6
7  @ClassAnnotation(type=BUSINESSCLASS,
    ↪ adaptationLevel=CLASSES, semantics="")
8  @Entity()
9  public entity Address {
10     //...
11 }
12
13 @ClassAnnotation(type=BUSINESSCLASS,
    ↪ adaptationLevel=CLASSES, semantics="")
14 @Entity(inheritanceType = SINGLE_TABLE)
15 public entity Person {
16     @Id(strategy = IDENTITY)
17     private int id;
18     private TString name;
19     @Relationship(type=ONE_TO_ONE, fetch=LAZY)
20     private Contact contact;
21     @Relationship(type=ONE_TO_MANY, fetch=LAZY)
22     private TList<Address> address;
23 }
24
25 @ClassAnnotation(type=BUSINESSCLASS,
    ↪ adaptationLevel=CLASSES, semantics="")
26 @Entity()
27 @Discriminator(value = "customer")
28 public entity Customer extends Person{
29     @Column(length = 14)
30     private TString individualRegistration;
31 }

```

LISTING 3. Customer entity's source code.

As shown in Listing 3, the entities were modeled based on adaptation annotations (`@ClassAnnotation`) and persistence annotations (`@Entity()`). The `@ClassAnnotation` annotations (lines 1, 7, 13, and 25) mean that these entities can be adapted at the class level (`adaptationLevel=CLASSES`), enabling their structure and/or behavior to be modified at runtime. The `@Entity()` annotations (lines 2, 8, 14, and 26) denote that these entities must be stored in a database. In line 14 it is possible to observe that the `Customer` entity was modeled based on the `SINGLE_TABLE` strategy, resulting in just one table in the database to deal with the `Customer` and `Person` entities. As reported in the description of the **Persistence Annotation Module**, the annotations are fully compliant with ORM persistence frameworks supported by JPA.

Finally, the **EntityModel package** represents the “main structure” of the **Core module**, since contains a model that enables the configuration of a project in the `DSLModeler4SaS`

tool, besides coordinating calls to other packages so that the design activity can be conducted by the developers. Fig. 8 illustrates the class diagram for the configuration of projects in the `DSLModeler4SaS` tool.

Regarding the structure, the starting of the model shown in Fig. 8 is the (`Project` class), which has relationships with entity packages (`EntityPackage` class), templates (`Template` class), libraries (`Library` class), and configuration of type mapping (`Mapping` class). The `Library` and `Mapping` classes work cooperatively, enabling developers to import libraries to map independent types to the target programming language. Regarding the design, this model expresses the relationships between two modeling elements of a project, namely: software entities and source code templates. Similar to the metamodel of the **Reflection Module** (see Fig. 5), the `Entity` class has a set of relationships (gray area) so that entities to be modeled concerning the structure (i.e., `EntityPackage`, `FieldEntity`, `MethodEntity`, and `Constructor` classes) and type mapping (i.e., `Mapping` class), both necessary to work with the approach of adaptation defined in `RA4SaS`. The relationships with blue classes represent the configuration of source code templates, such as the number of templates (`TemplateInfo` class), template scope (`TemplateTarget` enum), and sequence and frequency (`TemplateApplyMode` enum) in which these templates will be used by the **Source Code Module** to generate the software entities.

Concerning operational aspects, this model (Fig. 8) is used in two functionalities in the `DSLModeler4SaS` tool. First, this model is used to synchronize the files between a project modeled on the tool (i.e., in memory) and the hard disk of a computer. Second, this model is converted to the **Reflection Module** metamodel so that the source code of the SaS that is designed can be generated by the **Source Code Module**. It is worth mentioning that this conversion operation is performed by an internal parser of the *Core Module* (**EntityModel package**), enabling the **Source Code Module**, initially designed for `RA4SaS`, to be reused in the entity design scenario by the `DSLModeler4SaS` tool.

IV. CASE STUDY

To evaluate the applicability, strengths, and weaknesses of our architecture (i.e., `RA4SaS`, automated process for adaptation of the software entities, and the `DSLModeler4SaS` tool), this section presents a case study we have conducted to illustrate some adaptation scenarios. As a subject application for our empirical analysis, we have selected an application addressed to the management of a car rental. Next, a brief description of our subject application and the empirical strategies adopted for conducting this case study is reported.

Subject Application: Rent-a-Car is a web application organized in three layers: (i) Presentation; (ii) Application; and (iii) Database. For space and scope reasons, we will present in this article details of the **Application Layer** concerning the design, development, and adaptation of the software

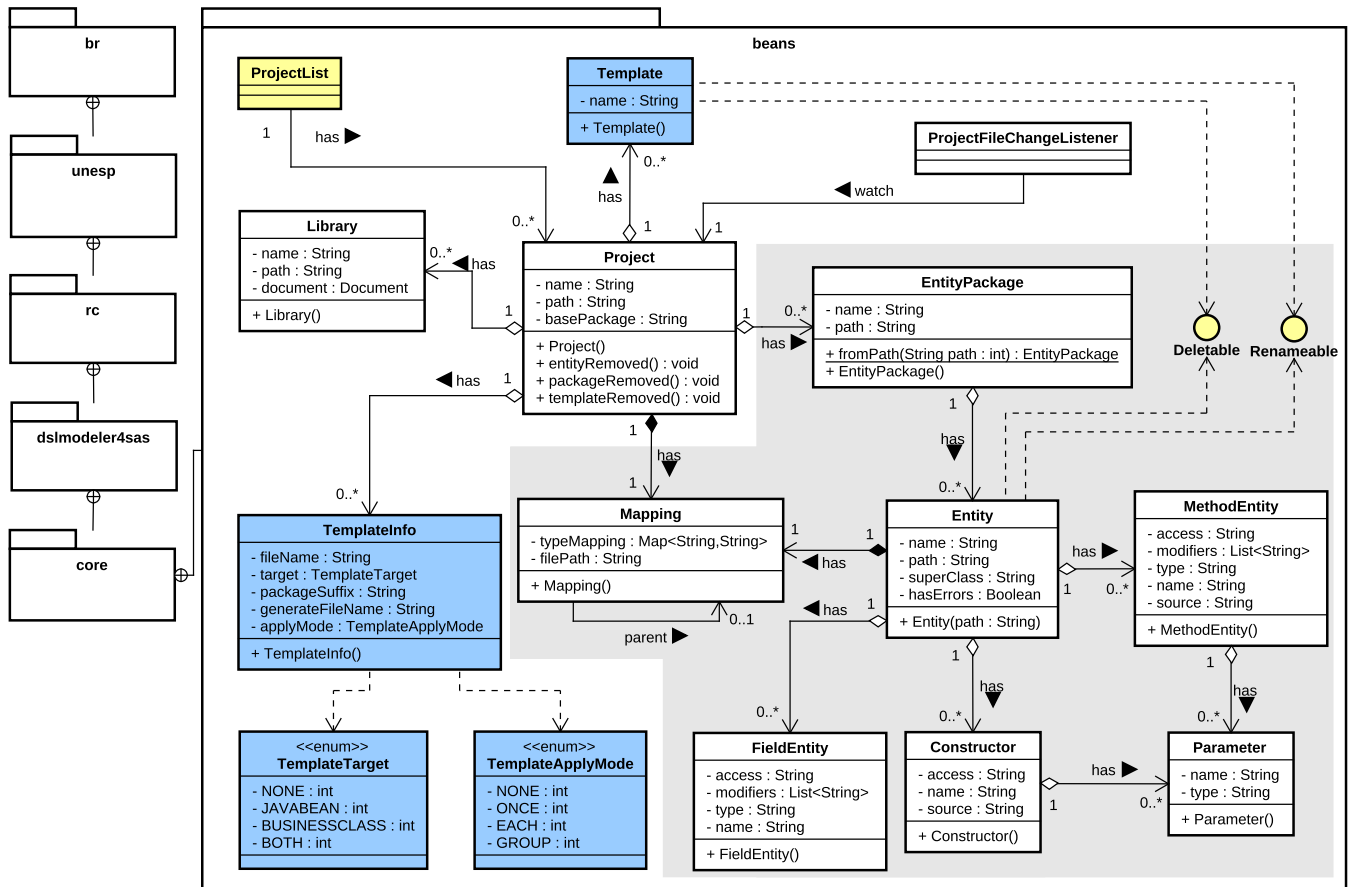


FIGURE 8. UML model for the configuration of projects in the DSLModeler4SaS tool.

entities. As reported in Section III-B6, our architecture has a tool called DSLModeler4SaS, which enables us to model software entities in a text-to-text approach through a DSL called eLanguage. Software engineers can focus their efforts on designing software entities and use RA4SaS's automated process to generate source code in a target programming language. To do so, they must work on the architectural design of an application based on design patterns and architectural decisions. As a result, they must create a set of templates for automatic code generation based on an organization model, as illustrated in Fig. 9.

Regarding the organization, the **Application layer** represents an architectural model composed of four packages, namely: `entity`, `service`, `dao`, `provider`, `config`. Note that each package has one or more files with the extension “.ftl”, which represent the templates designed in the design phase for generating application code in a target programming language by the **Source Code Module** of our architecture. Regarding the elaboration of these templates, it is worth mentioning that we must create them based on a target programming language (e.g., Java Programming Language) and a set of parameterized inputs. These entries must be filled with the information in each entity so that source code is generated in different layers (see **Source**

Code Module – Section III-B5). Regarding “.ftl” files, we have used a template engine called Freemarker¹⁰ in previous experiences of our research group [20]. In terms of design, these templates were developed based on the Hibernate ORM framework [38], which enables the mapping of object-oriented software to relational databases. In the empirical analysis, the software entities (i.e., SaS) were modeled in the DSLModeler4SaS tool through the DSL eLanguage and automatically generated by the **Code Source Module**. Once these software entities are inserted into the execution environment, they can generate their corresponding database entities following the persistence annotations (**Persistence Annotation Module**) inserted during the design phase. For reasons of scope and space, the source code of these templates will not be presented in this article. Next, we report a description of each package and its templates:

- The `entity` package just contains a `entity.ftl` file, which is the template responsible for generating the application's logical classes;
- In the `service` package, we inserted classes to address the business rules of the Rent-a-Car application.

¹⁰<https://freemarker.apache.org>

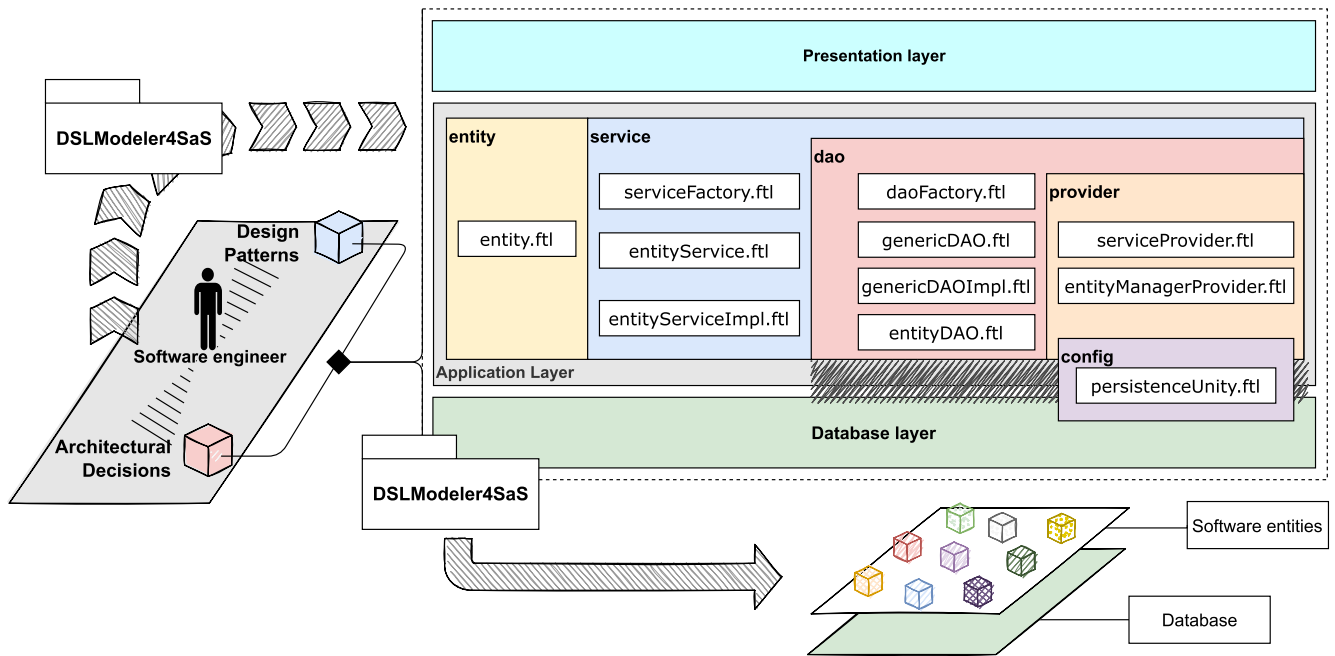


FIGURE 9. Architectural design in the DSLModeler4SaS tool.

The `entityService.ftl` and `entityServiceImpl.ftl` files are templates responsible for generating the interfaces and implementations of rule business operations. The file `serviceFactory.ftl` is an object factory of this package through the class instance by the interface. According to Gamma et al. [43], this strategy enables the implementation to be adapted without changing its use of the target class;

- The `dao` package contains some templates for generating the persistence source code of each entity. The `genericDAO.ftl` and `genericDAOImpl.ftl` files are used to generate the generic operations of persistence (i.e., CRUD¹¹ operations) for any class/entity. The `entityDAO.ftl` file enables the generation of persistence classes for each logical class (i.e., entities in DSL format contained in the `entity` package). The DAO classes inherit from the generic class so that its generic operations can be reused. Moreover, we can implement specific operations in each class generated by the `entityDAO.ftl` template. Finally, the `daoFactory.ftl` file is a template for an object factory of this package for injection of DAO classes into classes of the `service` package;
- The `provider` package is composed of the `serviceProvider.ftl` and `entityManagerProvider.ftl` files, which are templates for generating the persistence configuration unit and the persistence provider; and
- The `config` package just contains the `persistenceUnit.ftl` file, which is the template

responsible for generating the configuration file between the database layer and the application layer.

As a way to minimize the complexity of presenting our case study, Fig. 10 illustrates only the project of the `Customer` entity in the DSLModeler4SaS tool. In the project tree called `RentCar` it is possible to observe the source code of the software entities in the `br.unesp.rc.rentcar.entity` package. In addition, it is also possible to see the templates used for code generation, according to the architectural model illustrated in Fig. 9. To do so, software engineers must configure the templates, as illustrated in the Template Settings screen. Next, we present a setting description of these templates by column:

- **Template File.** In this column are listed all templates for a project (e.g., `textttRentCar`). For each line of this column, software engineers must configure the behavior of each template in the next columns.
- **Apply Mode.** This column enables to configure the use of these templates by the **Source Code Module**, namely: (i) `None` indicates that the template is not used to generate the source code file; (ii) `Once` denotes that the template is used only once and without dependencies. This value is indicated to generate generic structures of an architectural model, which do not depend on the entities themselves; (iii) `Each Entity` denotes that the template must be used for each entity. This value is the most important type because it enables the generation of specific structures for each entity in the different layers of an architectural model; and (iv) `Group` indicates that the template must be used over

¹¹CRUD = Create, Read, Update, and Delete.

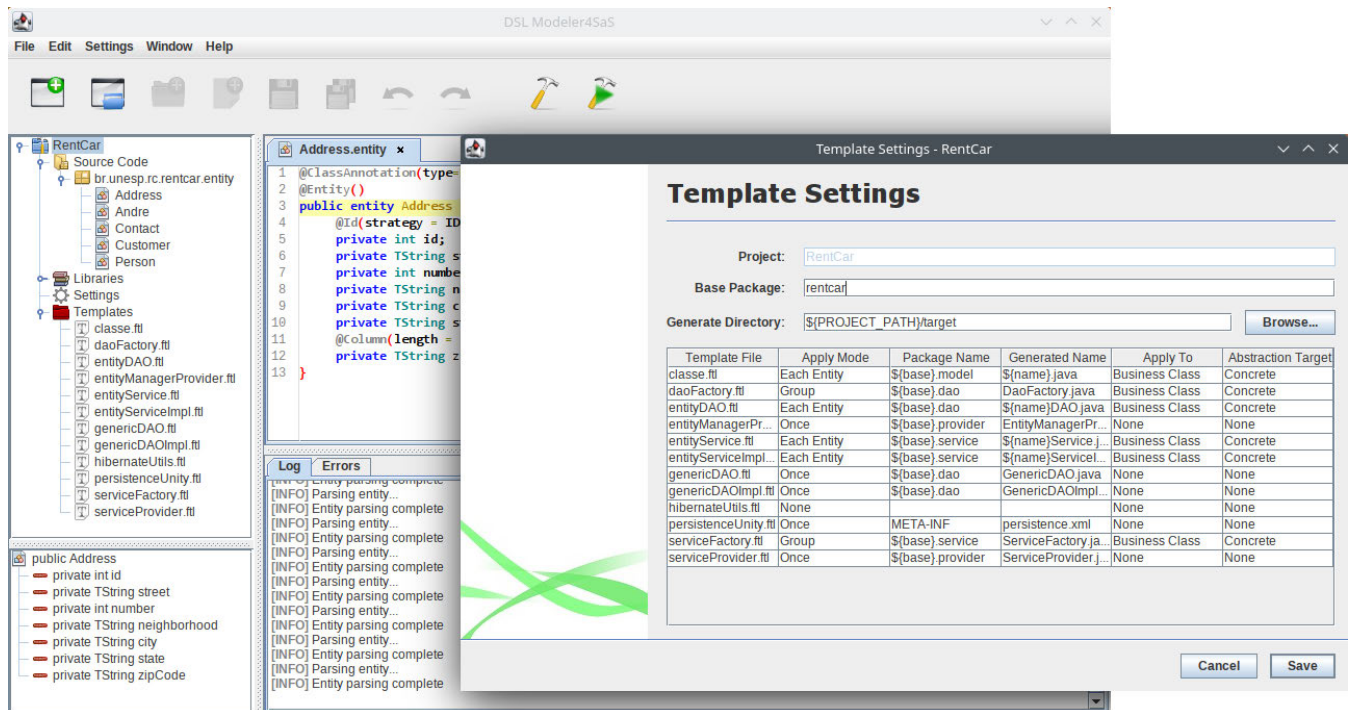


FIGURE 10. RentCar project in the DSLModeler4SaS tool.

all entities at once (e.g., enabling the construction of factories).

- **Package Name.** In this column, it is possible to configure the organization of templates concerning the target package (i.e., dao, model, provider, service, and META-INF), where “\$base.” represents the package with the project name (rentcar).
- **Generated Name.** This column enables the definition of the class names of an application according to architectural organizations (see Fig. 9), where “\$name” represents the name of the entity modeled in DSL format.
- **Apply To.** In this column, it is possible to define how the templates will be used concerning the type of entity that will be generated. To do so, None indicates that the template is not used, being more commonly used for new, deactivated, or of Once type (see Apply Mode column). JavaBean indicates that a template will only be applied over structures defined as a common class. Business Class indicates that we can apply a template over structures that are part of the business rule (e.g., entities that need to be persisted). Both indicates that we can apply a template to both types.
- **Abstraction Target.** This column enables the define the templates’ application concerning the type of abstraction of the entities, where None must be used for templates that must be executed only once, Concrete must be used for templates that need to derive classes from an entity set, Abstract must be used to templates that

should generate model entities but not business rules, and Both can be applied to previous ones.

As seen, not all items described for each column are present in the template setting. The setup illustrated in Fig. 10 shows the items used for this case study (i.e., Rent-a-Car application). Other systems may require a different setup for code generation by the **Source Code Module**.

Empirical Research Strategy: Based on the source code generated in the previous step (see Fig. 10), we selected three important adaptation scenarios at runtime to show the behavior and applicability of our RA, namely:

- 1) **Association of new functionalities**, which corresponds to the addition of new information (classes) to the selected software entity by aggregation, composition, or association;
- 2) **Extension of new functionalities**, which corresponds to the addition of new information (classes) to the selected software entity by the inheritance relationship; and
- 3) **Structural and behavioral adaptation**, which corresponds to the addition of new information (attributes and methods) to the selected software entity.

Before describing the aforementioned adaptation scenarios, two considerations must be reported. First, we instantiate our architecture in the Java¹² programming language (JDK 8) and we have not presented the implementation details in this article for space reasons. The second refers to the size

¹²<https://openjdk.org>

and logical organization of the chosen system, which can be characterized as an information system for managing a car rental. We have selected this system because it enables us to show the applicability of our architecture in some adaptation scenarios.

Fig. 11 illustrates the first two adaptation scenarios of our empirical research strategy. Step 1 represents the development of a software entity in the DSLModeler4SaS tool, as described in the “subject application” item of this section. Thus, from the code in DSL format, the Contact, Address, Person, and Customer entities are generated by the **Source Code Module** and inserted in the execution environment (Square A). We represent these entities through UML diagrams to differentiate entities in the design phase (i.e., DSL format) from those in the execution phase (class diagram). Note that we have annotated all entities as @ClassAnnotation, requiring at the same time class level adaptation (adaptationLevel=CLASSES) and storing information in a database (type=BUSINESSCLASS). Step 2 illustrates the first adaptation scenario, where an entity called Login is associated with Person through a composition relationship. This scenario shows that the Customer entity was designed to act in a local system, and subsequently adapted to act in a virtual system with authentication, but preserving the domain and goal of development. Regarding the adaptation steps, it is worth noting that the process illustrated in Fig. 6 must be used to carry out the adaptation activities. For scope and space reasons, only the code snippet showing the Login entity’s creation and its association with the Person entity is presented, as shown in Listing 4.

Initially, a Customer entity and Project class of the **Reflection Module** are instantiated (lines 1 and 4). The comments in lines 2 and 4 represented a set of omitted commands to instantiate each entity/object. In line 5, the customer entity is added to a project so that the metamodel of this entity can be instantiated (lines 25 to 29). The Login entity is created between lines 7 to 23. To create the username attribute for this entity is necessary to define an access modifier, data type, and attribute name (lines 7 to 10). Similarly (lines 12 to 15), the password attribute is created. Next, lines 17 to 19, these attributes must be inserted into a list of FieldClass so that they can be added to the Login entity. The Login entity is created between lines 21 and 23, where its name is defined in line 22 and its attributes are added in line 23. Between lines 25 and 29, the customer entity’s metamodel is generated by the ClassReflect class (**Reflection Module**). Finally, line 31 the Login entity is associated with the Person through a composition relationship. The first parameter (loginEntity) represents the entity that is being created (i.e., Login). The second (Person) and third (ONE-to-ONE) parameters show the entity name (metamodel) that will be associated with the new entity (first parameter) and the multiplicity between these entities, respectively. The fourth parameter (CP-UN-2) represents

```

1 Customer customer = new Customer();
2 //...
3 Project project = new Project();
4 //...
5 project.addObject(customer);
6
7 FieldClass username = new FieldClass();
8 username.setModifierQualifier("private");
9 username.setType("String");
10 username.setName("username");
11
12 FieldClass password = new FieldClass();
13 password.setModifierQualifier("private");
14 password.setType("String");
15 password.setName("password");
16
17 List<FieldClass> fields = new ArrayList<>();
18 fields.add(username);
19 fields.add(password);
20
21Clazz loginEntity = newClazz();
22 loginEntity.setName("Login");
23 loginEntity.setFields(fields);
24
25 try {
26     ClassReflect.fromProject(project);
27 } catch (Exception ex) {
28     System.out.println("Disassemble failed. " +
29         ↪ ex.getMessage());
30 }
31 project.addClazz(loginEntity, "Person",
32     ↪ "ONE-to-ONE", "CP-UN-2");

```

LISTING 4. Main adaptation commands.

the type of relationship (composition) and navigability (unidirectional) of this relationship (i.e., Person to Login). The UML model (Square B) shows the new Customer entity.

Step 3 represents the transition of the new Customer entity (Square B) to the initial adaptation stage. Thus, starting from the new entity (Square C), Step 4 illustrates the second adaptation scenario, where the new entity, initially designed to act in a software domain for a car rental, must be adapted to act in a domain of school management. To do so, a Student entity must be created and associated with Customer through an inheritance relationship, as illustrated in Fig. 11. The adaptation of these entities (Customer to Student) will not be presented at the same detailed level as the previous one, once the functioning and execution of commands are similar, as shown in Listing 5.

Initially, lines 3 to 6, the academicRecord attribute is created, as well as its access modifier, data type, and name. Next, this attribute is inserted into the field list (line 9) so that a Student entity can be created (lines 11 to 13). Based on the Customer entity (Square C), it must instantiate

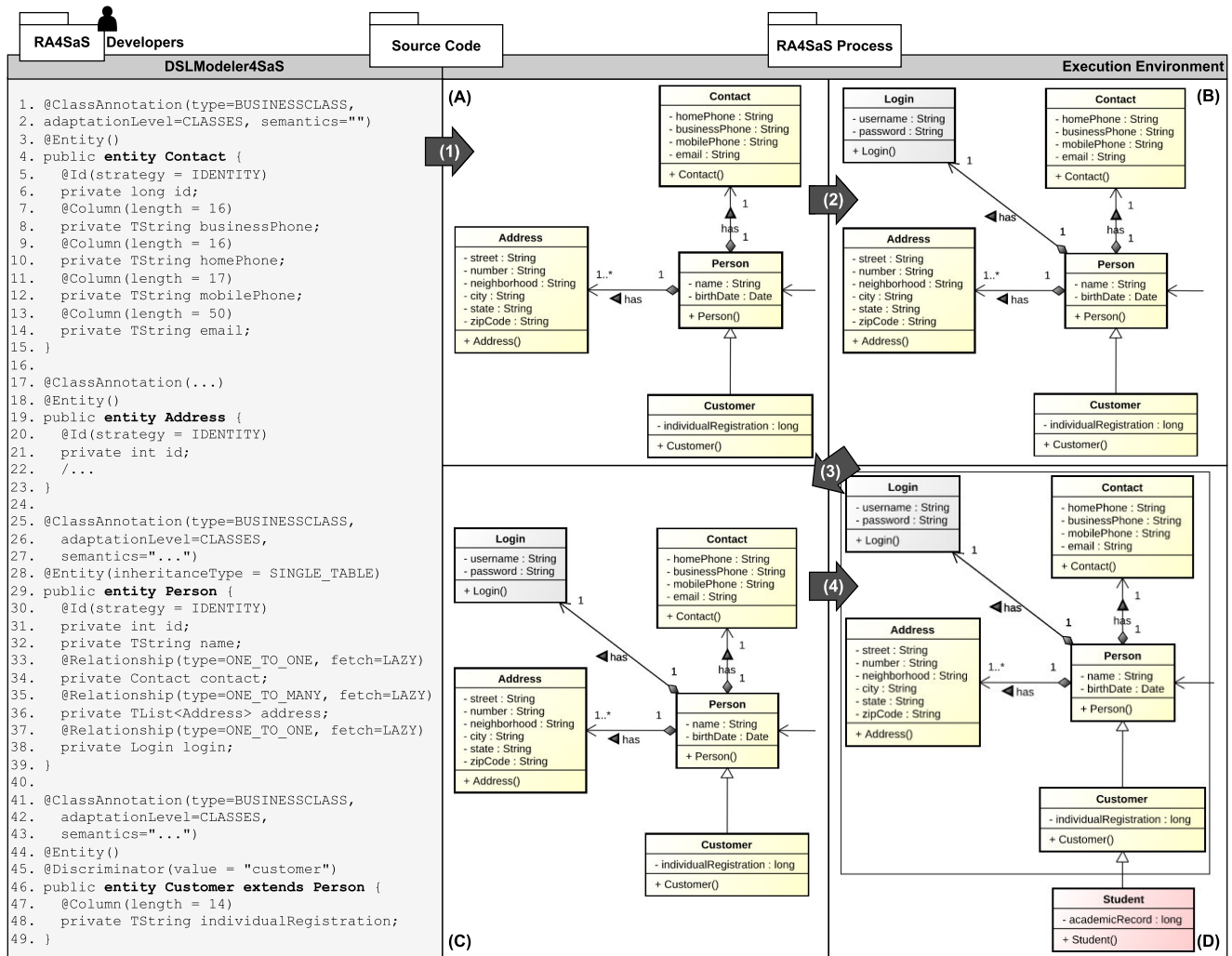


FIGURE 11. Adaptation of Customer entity for association and extension of new functionalities.

a metamodel with structural and behavioral information of this entity (lines 15 to 19). Line 21 shows the command for the relationship between the **Person** and **Student** entities. The first parameter (**studentEntity**) is the entity created to meet the second adaptation scenario. **Customer** is the second parameter of the command reported in line 21, which represents the name of the entity in the metamodel that will be associated with the new entity (i.e., the first parameter). The third (**NONE**) and fourth (**EXTENDS**) parameters show the multiplicity and the type of relationship (inheritance) between these entities do not exist, respectively. Square D shows the UML model of the **Student** entity.

Once the adaptation activities illustrated in Fig. 11 have been completed, the **Customer** entity (see Squares A, B, and D) can be utilized in three distinct execution scenarios. Fig. 12 illustrates each of the software entities and their respective databases (i.e., data schemas and data). Square A (**Customer**) represents the original entity, while Squares

B (**Customer'**) and D (**Student**) represent the first two adaptation scenarios (i.e., association of new features and extension of new features), respectively. As these entities were designed based on an ORM framework, the data schema was generated according to the annotations (**Persistence Annotation Module**) inserted in the design phase. This ensures that the software entities are equivalently mapped to database entities. In addition to the automated generation of the database's data schema, the utilization of an ORM framework facilitates the reuse of data persistence operations, thereby reducing the burden on developers during the development process.

Concerning the execution of the entities illustrated in Fig. 12 within the execution environment, two considerations must be highlighted. The first refers to the purpose of our case study, which aimed to present the potential of RA4SaS in its new release for the design of software entities through the DSLModeler4SaS tool with the DSL eLanguage and automatic code generation based on templates. The

```

1  // Omitted commands
2
3  FieldClass academicRecord = new FieldClass();
4  academicRecord.setModifierQualifier("private");
5  academicRecord.setType("long");
6  academicRecord.setName("academicRecord");
7
8  List<FieldClass> fields = new ArrayList<>();
9  fields.add(academicRecord);
10
11 Clazz studentEntity = new Clazz();
12  studentEntity.setName("Student");
13  studentEntity.setFields(fields);
14
15  try {
16      ClassReflect.fromProject(project);
17  } catch (Exception ex) {
18      System.out.println("Disassemble failed. " +
19          ↳ ex.getMessage());
20  }
21  project.addClazz(studentEntity, "Customer",
22      ↳ "NONE", "EXTENDS");

```

LISTING 5. Main adaptation commands.

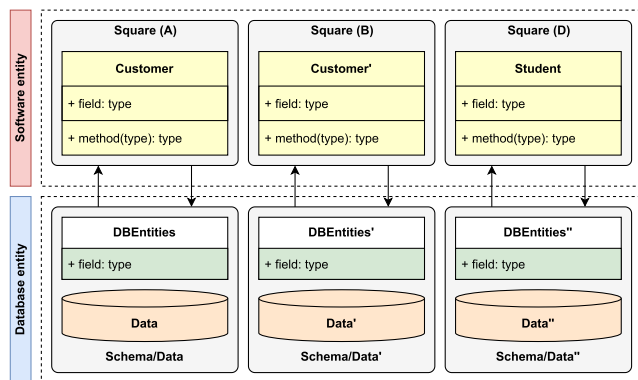


FIGURE 12. Software entities after adaptation.

second pertains to the functioning of these entities within the execution environment, where an adaptation operation may result in a change in the database (i.e., schema change), which may require data migration between schemas. In both adaptation scenarios (Squares B and D), a software entity was added, resulting in the addition of an equivalent entity in the database. Regarding execution, it can be stated that the three entities (Squares A, B, and D) remained active within the execution environment. In a real-world execution scenario, it is necessary to manage the lifecycle of these entities in order to ensure that, when there is no longer a reference to an entity, the data is migrated between schemas. However, for reasons of scope and complexity, this article will not present data schema and data evolution operations.

Fig. 13 illustrates the third adaptation scenario reported in this article. To show how this scenario occurs, two software

entities will be considered, namely: Car and Category. We will not present again the steps described in the previous scenarios for this one to avoid information redundancy. Initially, the software engineer must create the software entities in the DSLModeler4Sas tool in DSL format. Then, Step A, these entities are generated by the **Source Code Module** and inserted into the execution environment (see UML models). Step B illustrates the type of structural adaptation, where the `licensePlate` attribute (blue color) was inserted in the Car class. From the entity resulting from Step C, the `validateFormat` method was also inserted in the Car class, characterizing a type of behavioral adaptation (Step D). This method aims to verify if a particular license plate is in a specific format. To do so, we consider the two current formats in Brazil, namely: BRA-2022 and BRA2E22. The first format consists of three letters, a dash, and four digits. The second format is composed of three letters, one digit, one letter, and two digits.

Listing 6 shows the main commands for the structural and behavioral adaptation scenarios illustrated in Figure 13. As the scenarios do not occur in parallel, we present the commands for structural adaptation in lines 2 to 16, and the commands for behavioral adaptation in lines 19 to 35. The metamodel of the Car entity is instantiated between the ranges of lines 10 to 14 for structural adaptation and line 29 to 33 for behavioral adaptation. In lines 2 to 8, the `licensePlate` attribute is created and inserted in the list of attributes so that it can be added to the Car entity (line 16). Similarly, the `validateFormat` method is created between lines 21 and 24, and inserted in the method list (lines 6 and 27). For this structural scenario, the creation of a method involves defining a return type, a name for the method, and the source code. We choose not to present the source code of this method (line 19) because of the variety of implementations for this validation type. However, this method was stored in the source code repository in template format, being generated according to the attribute name (i.e., `licensePlate`).

After the third adaptation scenario (see Fig. 13), it is observed that the Car entity (original) underwent modification to align with the evolving structural and behavioral requirements. This scenario enables us to demonstrate the execution cycle of a software entity and its data schema from an alternative perspective. If on the one hand, behavioral adaptation does not affect the schema evolution of a software entity. Conversely, incorporating the `licensePlate` attribute requires modifications to the data schema of the Car entity in order to accommodate the storage of this information, consequently leading to the creation of a new entity. Consequently, when there were no requests for the original entity, data from that entity's schema can be migrated to the new entity's schema. Therefore, as previously outlined in the preceding adaptation scenarios, issues pertaining to the evolution of the data schema and data will not be addressed in this article due to considerations of scope and complexity.


```

1  // Omitted commands
2  FieldClass licensePlate = new FieldClass();
3  licensePlate.setModifierQualifier("private");
4  licensePlate.setType("String");
5  licensePlate.setName("licensePlate");
6
7  List<FieldClass> fields = new ArrayList<>();
8  fields.add(licensePlate);
9
10 try {
11     ClassReflect.fromProject(project);
12 } catch (Exception ex) {
13     System.out.println("Disassemble failed. " +
14         ↪ ex.getMessage());
15 }
16 project.addField("Car", fields);
17
18 // Omitted commands
19 String sourceCode = "... source code...";
20
21 MethodClass validateFormat = new MethodClass();
22 validateFormat.setReturnType("boolean");
23 validateFormat.setName("validateFormat");
24 validateFormat.setSourceCode(sourceCode);
25
26 List<MethodClass> methods = new ArrayList<>();
27 methods.add(validateFormat);
28
29 try {
30     ClassReflect.fromProject(project);
31 } catch (Exception ex) {
32     System.out.println("Disassemble failed. " +
33         ↪ ex.getMessage());
34 }
35 project.addMethod("Car", methods);

```

LISTING 6. Main adaptation commands.

V. DISCUSSION OF RESULT AND LIMITATION

This section summarizes the main findings and discusses the relevance of this work to the following communities: software engineering, software architecture, and SaS. Next, we list the main findings and results as follows.

This article presented a new version of RA4SaS, a reflection-based reference architecture for software adaptation at runtime. To the best of our knowledge, RA4SaS was the first reference architecture that encompasses two activities at the same time: entity design and adaptation at runtime without the perception of their stakeholders. In summary, this architecture enables the development of software entities through a controlled adaptation approach. To do so, the software engineer must annotate each entity to denote the level of adaptation supported (i.e., structural, behavioral, or both). In addition, we designed this architecture based on an automated process formed by a

well-defined sequence of steps that enables an entity to be adapted at runtime and to preserve its current execution states, as well as its initial requirements and goals definitions. In the new version presented in this article, we can summarize the innovations of our architecture in two items:

- Development of a **Persistence Annotation Module** for entities that need to store information in databases. In short, software engineers can insert persistence annotations to map persistence entities and their relationships. As a way of smoothing the learning curve for developers, we designed this annotation module based on the specification models of existing persistence frameworks in the literature (i.e., JPA); and
- Development of a **tool named DSLModeler4SaS** and a **DSL called eLanguage**, both designed to facilitate the build of SaS (i.e., software entities) through a text-to-text approach. To enable this approach, we customized **Source Code Module** and incorporated it into DSLModeler4SaS so that we could design the software entities in a domain-independent language and generate them in a target programming language. Thus, software engineers can focus their efforts on the design of entities, design decisions, and architectural design. To do so, they can create a set of templates so that it can generate the source code in all layers of the system (i.e., SaS). Therefore, it can be said that this strategy minimizes testing efforts and optimizes the RA4SaS automated process, since we generated the software entities through automated processes without involuntary injection of errors, favoring the adaptation activity.

According to different authors, software adaptation is an error-prone and onerous activity when performed manually [1], [2], [7], [10]. Therefore, automated processes are important for SaS development, since they increase the quick execution of tasks, minimizing time and cost, and reducing or eliminating the involvement of developers. Based on this scenario, it can be said that our architecture has become a comprehensive solution and a good perspective to contribute to the SaS area. As reported in Section III-B5, automated processes are present in two phases of our architecture, namely: design and runtime. First, the DSLModeler4SaS tool enables us to design software entities in a text-to-text approach supported by the automatic generation of source code. Second, these entities can be adapted at runtime through a well-defined sequence of steps as an assembly line. This sequence empowers software engineers to work at a higher level of abstraction, particularly during the design phase. This eliminates the requirement for them to possess specific knowledge to make modifications to a software entity.

Since its first version, we have found studies in the literature that report software adaptation at runtime at different abstraction levels. As the lowest abstraction level, one can cite the ASM framework,¹³ which requires from the developers a

¹³<https://asm.ow2.io>

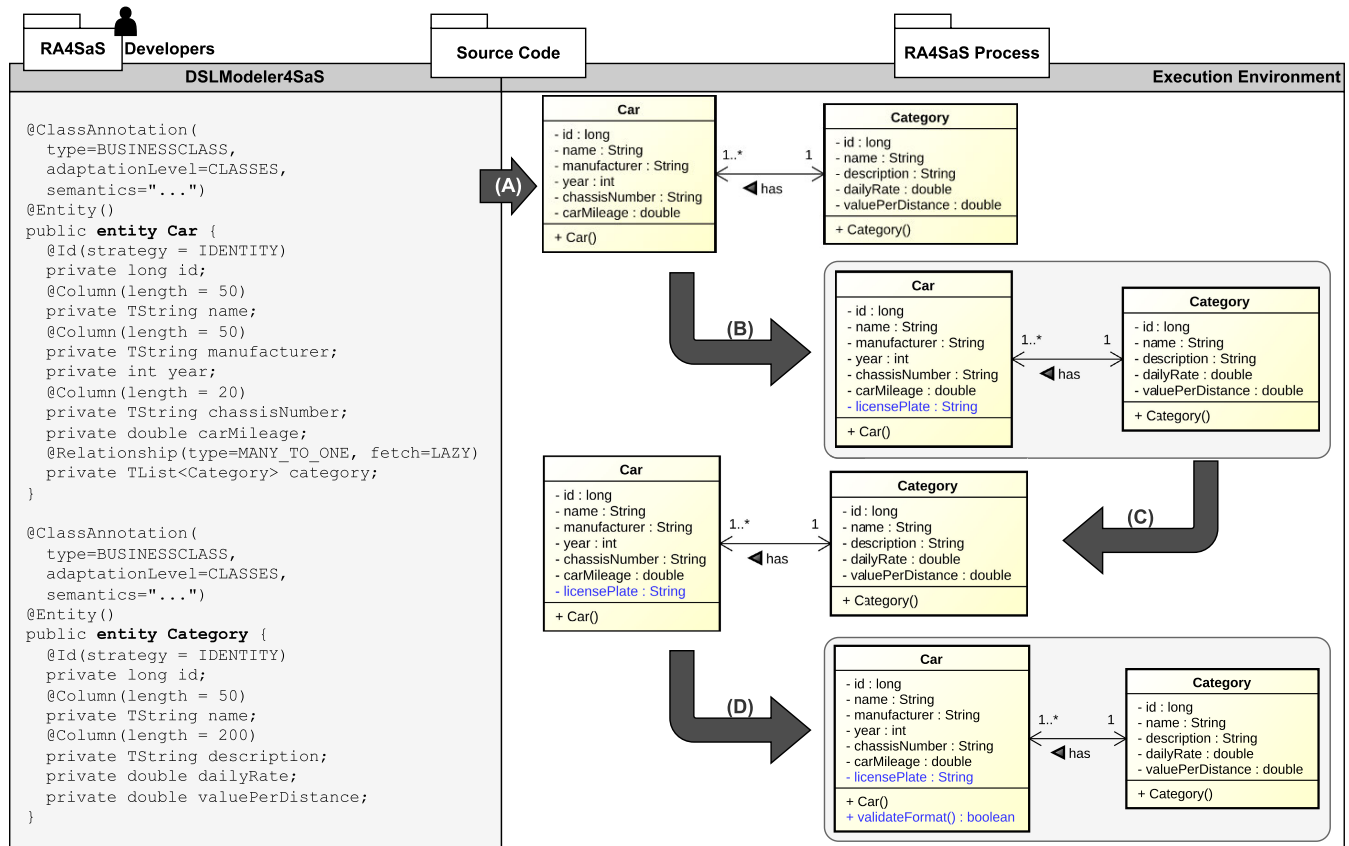


FIGURE 13. Structural and behavioral adaptation of **Car** entity.

specific level of knowledge for the manipulation in machine code. In contrast, our reference architecture enables software entities to be designed at a high abstraction level (i.e., DSL format) and adapted at runtime by an automated process. To do so, software entities are “disassembled” via reflection and inserted into a metamodel, which is changed and receives new structural and/or behavioral information for the new entity to be generated. Therefore, it can be said that software engineers can perform tasks more closely aligned with those they are accustomed to in their native development environment. Thus, our architecture has a good potential to become a facilitator for the development of SaS, since it provides a tool (DSLModeler4SaS and DSL) and an automated process that together can minimize the cognitive load of learning to develop this software type.

As reported in Section IV, we instantiated RA4SaS in the Java programming language. Although we have not yet instantiated our architecture in another programming language, we intend to evaluate its applicability and behavior in adapting software entities. Nevertheless, the results obtained at this moment lead us to believe that we can instantiate our architecture in other programming languages that support reflection.

The architecture presented in this article does not cover all cases and does not solve all problems related

to software adaptation at runtime. Software adaptation is a broad and complex issue because there are many involved details so that a software entity is adapted at runtime (e.g., specific features of adaptation, techniques of implementation, among others). Although the proposed architecture can be considered a generic solution, we have not investigated its applicability in different software domains (e.g., critical embedded systems). However, the results make us believe it can be extended or applied in this type of system since its guidelines can be adjusted (i.e., reused or adapted) to conduct the development of SaS automatically.

Finally, it can be said that RA4SaS paved the way for the design of RA4Self-MobApps [10] and RA4Self-CPS [16], which are examples of neighboring domain architectures. These architectures are aimed at supporting the development of mobile applications based on services that require runtime modification and self-adaptive cyber-physical systems, respectively. This scenario illustrates the achievement of knowledge sharing between systems in neighboring domains, as well as the reuse of this knowledge (i.e., architecture modules). Moreover, it can be highlighted that the process used during the design of these architectures (i.e., ProSA-RA [19]) has facilitated the transfer of knowledge and the expansion of SaS features to other software domains, such as service and CPS. Thus, this scenario provides promising

prospects for the reuse or extension of RA4SaS to other software domains.

VI. CONCLUSION AND FUTURE WORK

This article presented the second release of RA4SaS, a reference architecture that intends to support the development of SaS through a tool named DSLModeler4SaS, a DSL called Language, and an automated process for adaptation at runtime. In short, this architecture enables the development of software entities through the controlled adaptation approach. To do so, software engineers must insert annotations to define the adaptation level of each entity, besides annotations for entities that require data persistence. At runtime, our architecture enables the preservation of the execution states of each entity, besides controlling the size, requirements, and goals of these entities so that future adaptations do not become unfeasible. Based on this scenario, the main contributions of this article are:

- According to the best of our knowledge, we have proposed the first reference architecture that enables the development of SaS supported by a tool named DSLModeler4SaS and a text-to-text approach through a DSL called eLanguage, which can be a good contribution to the area of software architecture because our architecture can be used in the design of other RAs;
- In the realm of the SaS area, we contribute by offering a means to design and adapt software entities at runtime without the perception of their stakeholders. The DSLModeler4SaS tool, along with the automated runtime adaptation process, streamlines the design tasks for software entities and their adaptation in the execution environment. Moreover, this tool assists in entity development and automatic code generation, ensuring error-free structures and operations derived from the templates. Consequently, the quality of the generated code enhances the efficiency of automated processes.
- In the field of software engineering, our architecture enables developers to create software entities using a text-to-text approach. To do so, developers must first create templates based on design patterns and architectural decisions. They can then design the software entities in DSL format and generate the system source code in the target programming language chosen during the design phase. As a result, developers can concentrate their efforts on the design aspects of software entities, as all repetitive tasks are automated by the tool; and
- In the different areas, our architecture may effectively complement established software development processes commonly used by companies in various domains. The RA4SaS, including the DSLModeler4SaS tool, Language DSL, and automated processes, appears to align well with existing practices, suggesting compatibility and potential synergies between them.

Regarding future work on RA4SaS, we intend to conduct at least three activities, namely: (i) conduction of more case

studies or experiments intending to evaluate our architecture, including different software domains; (ii) instantiation of our architecture for other programming languages to evaluate its structures and respective elements, behavior, and relationships between them when a concrete architecture is instantiated; and (iii) use of our architecture in the industry to evaluate its behavior when it is applied in a larger real environment of development and execution. Thus, based on the content presented in this article, a good research perspective can be idealized, since our architecture has excellent potential to become an effective contribution to both communities: software engineering, software architecture, and SaS.

REFERENCES

- [1] M. Salehie and L. Tahvildari, "Self-adaptive software: Landscape and research challenges," *ACM Trans. Auto. Adapt. Syst.*, vol. 4, no. 2, pp. 1–42, May 2009.
- [2] B. H. C. Cheng et al., "Software engineering for self-adaptive systems: A research roadmap," in *Software Engineering for Self-Adaptive Systems*, B. H. C. Cheng, R. de Lemos, H. Giese, P. Inverardi, and J. Magee, Eds., Berlin, Germany: Springer, 2009, pp. 1–26.
- [3] R. de Lemos et al., "Software engineering for self-adaptive systems: A second research roadmap," in *Software Engineering for Self-Adaptive Systems II: International Seminar*, R. de Lemos, H. Giese, H. A. Müller, and M. Shaw, Eds., Dagstuhl Castle, Germany: Springer, Oct. 2013, pp. 1–32.
- [4] R. de Lemos, D. Garlan, C. Ghezzi, and H. Giese, "Software engineering for self-adaptive systems III. Assurances," in *Software Engineering for Self-Adaptive Systems III. Assurances International Seminar, Dagstuhl Castle, Germany, December 15–19, 2013, Revised Selected and Invited Papers*. Cham, Switzerland: Springer, 2017.
- [5] D. Weyns, "Software engineering of self-adaptive systems," in *Handbook of Software Engineering*, S. Cha, R. N. Taylor, and K. Kang, Eds., Cham, Switzerland: Springer, 2019, pp. 399–443.
- [6] E. Borde, G. Haik, and L. Pautet, "Mode-based reconfiguration of critical software component architectures," in *Proc. Design, Autom. Test Eur. Conf. Exhib.*, Leuven, Belgium, Apr. 2009, pp. 1160–1165.
- [7] C. Krupitzer, F. M. Roth, S. VanSyckel, G. Schiele, and C. Becker, "A survey on engineering approaches for self-adaptive systems," *Pervas. Mobile Comput.*, vol. 17, pp. 184–206, Feb. 2015.
- [8] F. D. Macías-Escrivá, R. Haber, R. del Toro, and V. Hernandez, "Self-adaptive systems: A survey of current approaches, research challenges and applications," *Expert Syst. Appl.*, vol. 40, no. 18, pp. 7267–7279, Dec. 2013.
- [9] W. F. Passini and F. J. Affonso, "Developing self-adaptive service-oriented mobile applications: A framework based on dynamic deployment," *Int. J. Softw. Eng. Knowl. Eng.*, vol. 28, no. 11n12, pp. 1537–1558, Nov. 2018.
- [10] F. J. Affonso, W. F. Passini, and E. Y. Nakagawa, "A reference architecture to support the development of mobile applications based on self-adaptive services," *Pervas. Mobile Comput.*, vol. 53, pp. 33–48, Feb. 2019.
- [11] N. Bencomo and G. Blair, "Using architecture models to support the generation and operation of component-based adaptive systems," in *Software Engineering for Self-Adaptive Systems*, B. H. C. Cheng, R. de Lemos, H. Giese, P. Inverardi, and J. Magee, Eds., Berlin, Germany: Springer, 2009, pp. 183–200.
- [12] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed., Boston, MA, USA: Addison-Wesley, 2012.
- [13] L. Liu, S. Thanheiser, and H. Schmeck, "A reference architecture for self-organizing service-oriented computing," in *Architecture of Computing Systems—ARCS*. Berlin, Germany: Springer, 2008, pp. 205–219.
- [14] D. Weyns, S. Malek, and J. Andersson, "FORMS: A formal reference model for self-adaptation," in *Proc. 7th Int. Conf. Autonomic Comput.*, New York, NY, USA, Jun. 2010, pp. 205–214.
- [15] F. J. Affonso and E. Y. Nakagawa, "A reference architecture based on reflection for self-adaptive software," in *Proc. VII Brazilian Symp. Softw. Compon., Architectures Reuse*, Sep. 2013, pp. 129–138.

- [16] M. P. D. O. Camargo, G. D. S. Pereira, D. Almeida, L. A. Bento, W. F. Dorante, and F. J. Affonso, "RA4Self-CPS: A reference architecture for self-adaptive cyber-physical systems," *IEEE Latin Amer. Trans.*, vol. 22, no. 2, pp. 113–125, Feb. 2024.
- [17] F. J. Affonso, K. R. F. Scannavino, L. B. R. Oliveira, and E. Y. Nakagawa, "Reference architectures for self-managed software systems: A systematic literature review," in *Proc. 8th Brazilian Symp. Softw. Compon., Architectures Reuse*, Sep. 2014, pp. 21–31.
- [18] F. J. Affonso, G. N. Campos, and G. G. Menaldo. (2023). *Mapping Study on Reference Architectures for Self-adaptive Software*. Accessed: Feb. 26, 2024. [Online]. Available: https://drive.google.com/file/d/1CnVuMR-LL3nfz7NjFNBtbmzLy9C_qBL9/view?usp=sharing
- [19] E. Y. Nakagawa, M. Guessi, J. C. Maldonado, D. Feitosa, and F. Oquendo, "Consolidating a process for the design, representation, and evaluation of reference architectures," in *Proc. IEEE/IFIP Conf. Softw. Archit.*, Apr. 2014, pp. 143–152.
- [20] G. S. Benato, F. J. Affonso, and E. Y. Nakagawa, "Infrastructure based on template engines for automatic generation of source code for self-adaptive software domain," in *Proc. Int. Conf. Softw. Eng. Knowl. Eng.*, Pittsburgh, PA, USA, Jul. 2017, pp. 30–35.
- [21] N. M. Villegas, G. Tamura, and H. A. Müller, "Architecting software systems for runtime self-adaptation: Concepts, models, and challenges," in *Managing Trade-Offs in Adaptable Software Architectures*, I. Mistrik, N. Ali, R. Kazman, J. Grundy, and B. Schmerl, Eds., San Mateo, CA, USA: Morgan Kaufmann, 2017, pp. 17–43.
- [22] R. Laddaga, "Active software," in *Self-Adaptive Software*, P. Robertson, H. Shrobe, and R. Laddaga, Eds., Berlin, Germany: Springer, 2001, pp. 11–26.
- [23] IBM. (2005). *An Architectural Blueprint for Autonomic Computing*. Accessed: Feb. 26, 2024. [Online]. Available: <https://www-03.ibm.com/autonomic/pdfs/AC>
- [24] P. Maes, "Concepts and experiments in computational reflection," in *Conf. Proc. Object-Oriented Program. Syst., Lang. Appl.*, New York, NY, USA, Dec. 1987, pp. 147–155.
- [25] S. Vinoski, "A time for reflection [software reflection]," *IEEE Internet Comput.*, vol. 9, no. 1, pp. 86–89, Jan. 2005.
- [26] J. Andersson, R. de Lemos, S. Malek, and D. Weyns, "Reflecting on self-adaptive software systems," in *Proc. ICSE Workshop Softw. Eng. Adapt. Self-Managing Syst.*, May 2009, pp. 38–47.
- [27] M. Pukall, C. Kästner, W. Cazzola, S. Götz, A. Grebhahn, R. Schröter, and G. Saake, "JavAdaptor—Flexible runtime updates of Java applications," *Softw., Pract. Exper.*, vol. 43, no. 2, pp. 153–185, Feb. 2013.
- [28] U. Aßmann, S. Götz, J.-M. Jézéquel, B. Morin, and M. Trapp, "A reference architecture and roadmap for Models@run.time systems," in *Models@run.time: Foundations, Applications, and Roadmaps*, N. Bencomo, R. France, B. H. C. Cheng, and U. Aßmann, Eds., Cham, Switzerland: Springer, 2014, pp. 1–18.
- [29] E. Y. Nakagawa, P. Oliveira Antonino, and M. Becker, "Reference architecture and product line architecture: A subtle but critical difference," in *Software Architecture (Lecture Notes in Computer Science)*, vol. 6903, I. Crnkovic, V. Gruhn, and M. Book, Eds., Berlin, Germany: Springer, 2011, pp. 207–211.
- [30] P. Kruchten, *The Rational Unified Process: An Introduction*, 3rd ed., Boston, MA, USA: Addison-Wesley, 2003.
- [31] M. Rosen, S. W. Ambler, T. K. Hazra, W. Ulrich, and J. Watson, "Enterprise architecture trends," *Cutter Consortium*, vol. 10, no. 1, 2007.
- [32] S. Angelov, P. Grefen, and D. Greefhorst, "A classification of software reference architectures: Analyzing their success and effectiveness," in *Proc. Joint Work. IEEE/IFIP Conf. Softw. Archit. Eur. Conf. Softw. Archit.*, Sep. 2009, pp. 141–150.
- [33] S. Angelov, P. Grefen, and D. Greefhorst, "A framework for analysis and design of software reference architectures," *Inf. Softw. Technol.*, vol. 54, no. 4, pp. 417–431, Apr. 2012.
- [34] K. Petersen, S. Vakkalanka, and L. Kuzniarz, "Guidelines for conducting systematic mapping studies in software engineering: An update," *Inf. Softw. Technol.*, vol. 64, pp. 1–18, Aug. 2015.
- [35] F. J. Affonso, G. N. Campos, and G. G. Menaldo. (2023). *Architectural Requirements*. Accessed: Feb. 26, 2024. [Online]. Available: https://drive.google.com/file/d/1MvRqEebNge478o9Fuehd4jILWB_HYcQX/view
- [36] T. Vogel. (2023). *Software Engineering for Self-Adaptive Systems. Exemplars Section. Self-Adaptive Systems Artifacts and Model Problems*. Accessed: Feb. 26, 2024. [Online]. Available: <https://www.hpi.uni-potsdam.de/giese/public/selfadapt/exemplars/>
- [37] F. J. Affonso and E. Y. Nakagawa, "Self-adaptive software: Development approach and automatic process for adaptation at runtime," *Revista Brasileira de Computação Aplicada*, vol. 7, no. 1, pp. 68–84, Apr. 2015.
- [38] Hibernate. (2024). *Hibernate ORM*. Accessed: Feb. 26, 2024. [Online]. Available: <https://hibernate.org/orm/>
- [39] F. J. Affonso, G. Leite, R. A. P. Oliveira, and E. Y. Nakagawa, "A framework based on learning techniques for decision-making in self-adaptive software," in *Proc. Int. Conf. Softw. Eng. Knowl. Eng.*, Jul. 2015, pp. 24–29.
- [40] F. Affonso, G. Leite, and E. Nakagawa, "DmS-modeler: A tool for modeling decision-making systems for self-adaptive software domain," in *Proc. Int. Conferences Softw. Eng. Knowl. Eng.*, Jul. 2016, pp. 617–622.
- [41] ISO/IEC/IEEE International Standard—Systems and Software Engineering—Vocabulary, IEEE Standard 24765:2017, 2017, pp. 1–541.
- [42] I. R. Forman and N. Forman, *Java Reflection in Action*. New York, NY, USA: Manning, 2004.
- [43] E. Gamma, R. Helm, R. Johnson, and J. M. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, 1st ed., Reading, MA, USA: Addison-Wesley, 1994.
- [44] ANTLR. (2024). *ANTLR (Another Tool for Language Recognition)*. Accessed: Feb. 26, 2024. [Online]. Available: <https://www.antlr.org>



FRANK JOSÉ AFFONSO received the Ph.D. degree from the University of São Paulo (USP), in 2009. From 2013 to 2014, he was a Postdoctoral Researcher in computer science with ICMC/USP. He is currently an Assistant Professor with São Paulo State University (UNESP), Rio Claro, Brazil. He has experience in computer science with an emphasis on software engineering in the following areas, such as reference architecture, service-based systems, mobile computing, and self-adaptive software systems. He advises students in Unesp's Graduate Program. He is a member of program committees at many conferences and has been a reviewer of several journals. He is the institutional representative of the Brazilian Computing Society (SBC) in Rio Claro.



GABRIEL NAGASSAKI CAMPOS received the master's degree in computer science from São Paulo State University (UNESP), in 2024. He was a Developer of web and desktop systems for sales management, aircraft sharing, and access control based on the Microsoft platform. His research interests include software architecture, software engineering, self-adaptive software, database systems, and web development.



GUILHERME GUIGUER MENALDO received the bachelor's degree in computer science from São Paulo State University (UNESP), in 2018. He is currently a Software Developer with IBM Corporation, Brazil (Hortolandia/São Paulo). He has experience in computer science with an emphasis on the following areas, such as software engineering, self-adaptive software, CASE Tools, service-oriented applications, database systems, and web development.

...